LONDON METROPOLITAN UNIVERSITY

# Module Code: CS5003

# Module Title: Data Structures and Specialist Programming

Assessment Component: Coursework (002)
Topic Title: Choc Cart

Academic Year: 2025/26
Module Leader: Mona Abdelgayed


Student Name - Student ID

- Aavash Sapkota - 22081679
- Krish Agrawal- 23046217
- Sandesh Bikram Shahi- 24015195
- Link GitHub: https://github.com/aavash15/Choco-shop-
-


These are the steps to run the project:

• First, download the project from the GitHub link.
• Open the Apache NetBeans IDE and select “Open Project”.
• Then locate the file and open the Chocoshopp project folder.
• Ensure that Java JDK (version 8 or above) is installed to run the project.
• Open the Chocoshop_DB.sql script provided in the root folder, select the new connection, and run the file to build the tables and insert the simple data.
• Confirm the web server is attached to the GlassFish in NetBeans.
• Update database connection details (username, password, database name) in the project files.
• Clean, build and run the project and open it in the localhost link.

# TABLE OF CONTENT

TABLE OF CONTENT ..... 1
1. INTRODUCTION ..... 2
1.1. Title: ..... 3
1.2. Purpose ..... 3
1.3. Audience: ..... 3
1.4. Notable Features: (front-end for the user) ..... 3
1.5. Implemented Functions: (back-end for the system) ..... 3
1.6. Additional function ..... 3
1.6 Technologies used: ..... 3
1.7. The aim of the project ..... 4
2. SITE MAP ..... 4
2.1. Breakdown of the pages. ..... 5
2.2. User Navigation Flow: ..... 5
2.3. Validation ..... 5
3. Discussion of Usability/Design Principles ..... 7
4.2. Relationship table ..... 11
5. TESTING AND IMPLEMENTATION: ..... 11
5.2 CRUD Operations ..... 13
5.2.1 Create ..... 14
5.3 Validation ..... 24
5.3.1 Registration Validation ..... 25
5.4 Ajax ..... 26
5.4.1 Cart Quantity Update (keyup event) ..... 26
5.4.2 Cart Item Remove (commandButton with Ajax) ..... 27
5.4.3 Clear Cart (commandButton with Ajax) ..... 27
5.5 Session Tracking ..... 28
5.6 UI Components ..... 29
5.7 Checkout and Payment Flow ..... 32
5.8 Order History ..... 33
6. ADDITIONAL FEATURES: ..... 33
7. WORK MANAGEMENT and REFLECTION ..... 34
7.1 Work distribution ..... 35
7.2. Refection ..... 35
8. ARTEFACT DESCRIPTION ..... 36

9. Reference .....38

# 1. INTRODUCTION

## 1.1. Title: Choco Cart (Online Chocolate Shopping Web Application)

## 1.2. Purpose: To provide an easy and enjoyable online shopping experience while buying chocolate for the buyers. This is an online and web application where buyers can buy chocolate from comfort.

## 1.3. Audience: The target audience includes chocolate enthusiasts, casual online shoppers, and gift buyers of all ages who prefer a convenient, browser-based purchasing experience.

## 1.4. Notable Features: (front-end for the user)

- Product browsing with chocolate details (Name, details, price)
- Basket management: session tracking along with add, delete, adjust, etc.
- Checkout process: sample checkout forum.
- Validation: Input validation during checkout and user login.
- User registration

## 1.5. Implemented Functions: (back-end for the system)

- Realtime basket updates
- Secure payment forum (beta)
- User login authentication

## 1.6. Additional function

- Two databases: one stores product stock and type; the other stores the sold record with order ID. More in Section 4.
- Order history for the -in user
- Auto-update stock after the payment by the users
- Check for duplicate emails during registration
- Password check during registration
- Tracking stock, the database
- Store old and sold-out items using separate order records.
- UI component
- Tooltips for the buttons.
- Different ways to pay. (karma, buy one pay later)
- Users can send the message to the admin via contact with us. (All the messages are saved in the database.)

## 1.6 Technologies used:

The Choco cart was developed as a Java **client-server web application** using JavaServer Faces **(JSF), JavaBeans, Ajax, session tracking, and a backend database.** The use of different types of functions in the project to produce the dynamic website. Particularly, JSF was used to create dynamic web pages and user interfaces. (Eclipse Foundation, 2022)

While handling the main application, logic, login, basket management and checkout processing are supported by the JavaBeans. (Oracle, 2024b)

Ajax was to provide real-time data without reloading the page. This helped them to stabilise the system. (Oracle, 2014a)

Session tracking is used for the user's logging state and shopping basket across the page. (Oracle, 2014b)

Code comments help us to understand the code and help us navigate the code faster. It was handy when we passed the code around during handover.

Backend database was the backbone of the project that stores product, user, and other data like order ID, items. It also supports database access and CRUD-related operations through the web application. (Oracle, 2024a)

Overall, using all these technologies allowed the system to fulfil the coursework requirement for the dynamic Java web application. This also highlights the database integration, validation, navigation, and finally, user interactive functions.

## 1.7. The aim of the project

is to generate realistic yet manageable e-shop applications that meet the core course requirements for some extra details. While we showcase the creative design part, we also demonstrate the core technical requirement of the project. Choco cart is aimed to provide the users with the simple yet interactive shopping experience through product browsing, basket management, login and registration, and checkout, supported by JSF, JavaBeans, Ajax, session tracking, validation, and backend database access.

# 2. SITE MAP

## 2.1. Breakdown of the pages.

The site map of **Choco Cart** is divided into small sections to make the structure of the website clear and easy to follow:

| Page | Type | Connects to | Key features |
|---|---|---|---|
| **Home Page** | Dynamic | All pages | Link to all section clean UI |
| **Register page** | Dynamic | Login page | New user account, password validation (JSF, JavaBeans) |
| **Login page** | Dynamic | Register page, Home page | Authentication, session tracking begins |
| **Product page** | Dynamic | Basket page | Chocolate information, add to cart, and pictures |
| **Contact Us page** | Static | Home page | Shop's information |
| **Basket page** | Dynamic | Login page, product page | Ajax-powered live, CRUD operators like add, delete, update, create new account |
| **Checkout page** | Dynamic | Login page, product page | Card details, login if you have an account if you do not create the new account. |
| **Order confirmation page** | Dynamic | Home page, continue shopping | Comfirms order place and return to home |

## 2.2. User Navigation Flow:

- The website is clean yet professional, so non-tech users can navigate easily throughout the website.
- A guest can browse the chocolate, see the live price, and add/change it to the cart/basket.
- They can proceed to the checkout page, which needs an account with Choco Cart, and the basket is preserved via session tracking. However, to move to checkout, they need to log in to their ID; if not, they need to create a new ID for the shop.
- After logging in, details of the card are entered on the checkout page to complete the order.

## 2.3. Validation

There are major validation parts.

- Password must follow:
  - the minimum 8 characters,
  - at least 1 uppercase,
  - 1 lowercase,
  - 1 number,
  - and 1 special character

  This improves security and reflects real-world patterns.

- While card validation checks:
  - The card name: string and card number: integer.
  - No null data,
  - Expiry data and CVV must be valid in format and length.
  - Valid billing address.

These check the validation, making beta testing stronger.

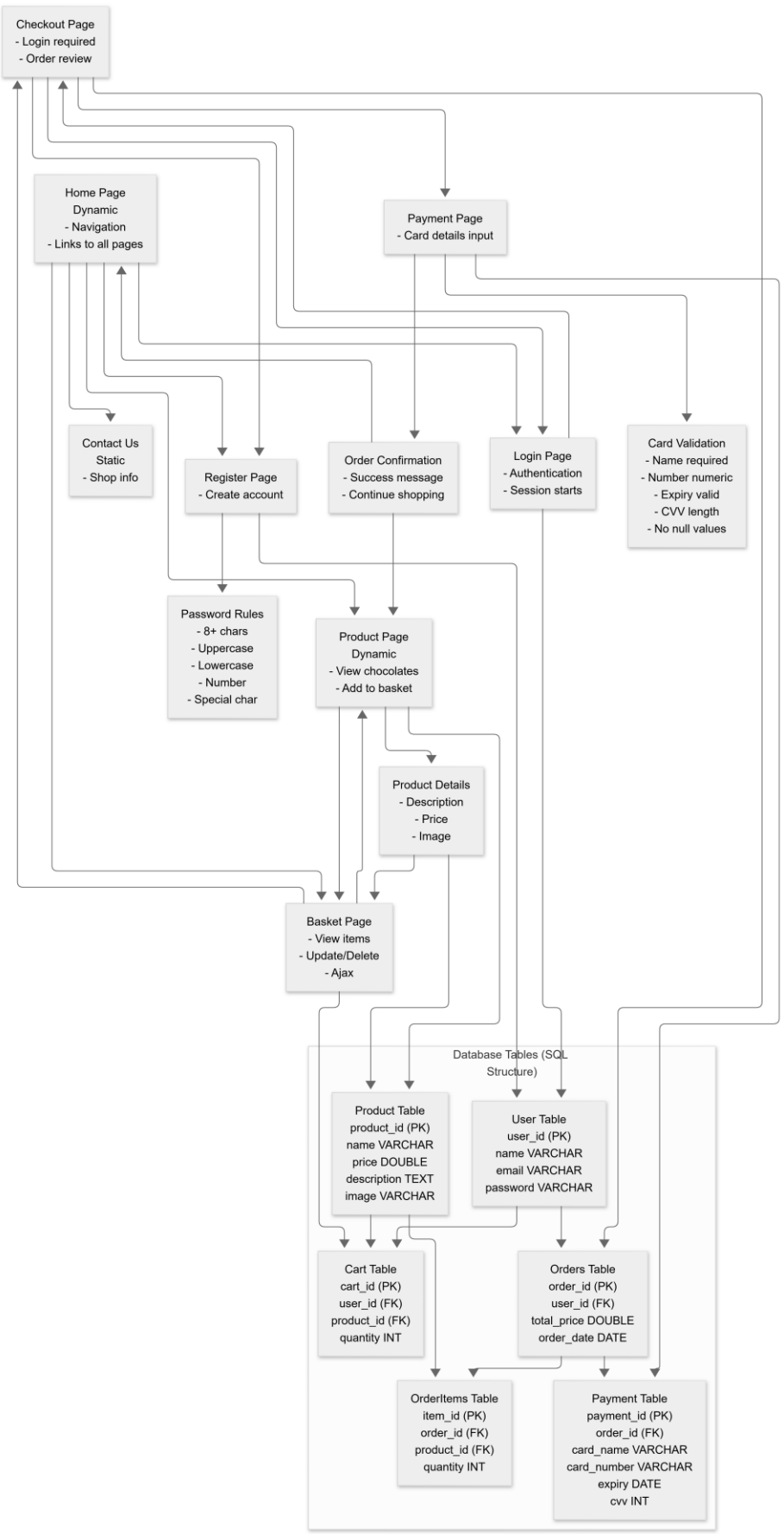



Fig 2. UML diagram for the project in details

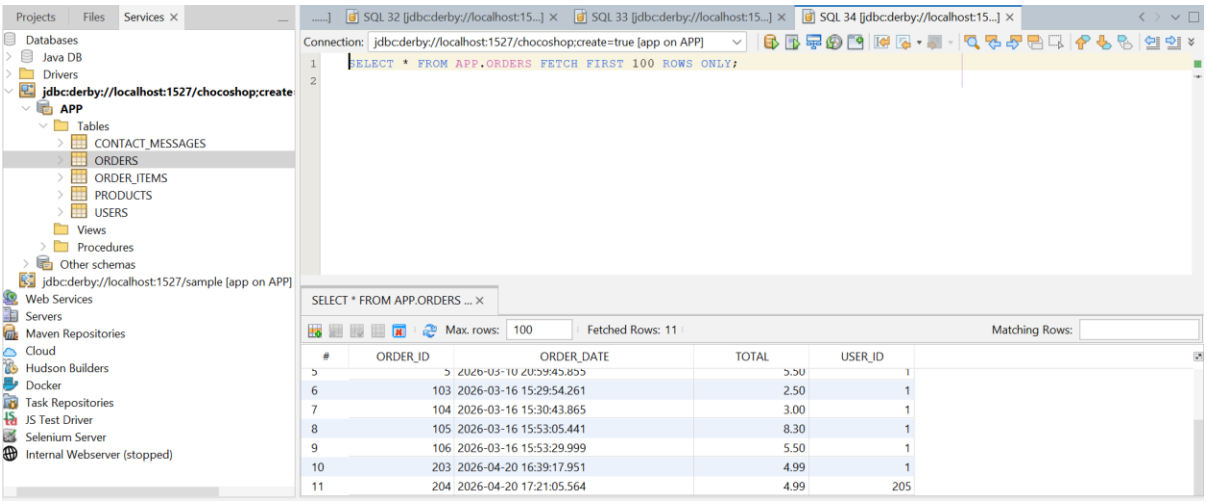
## 3. Discussion of Usability/Design Principles

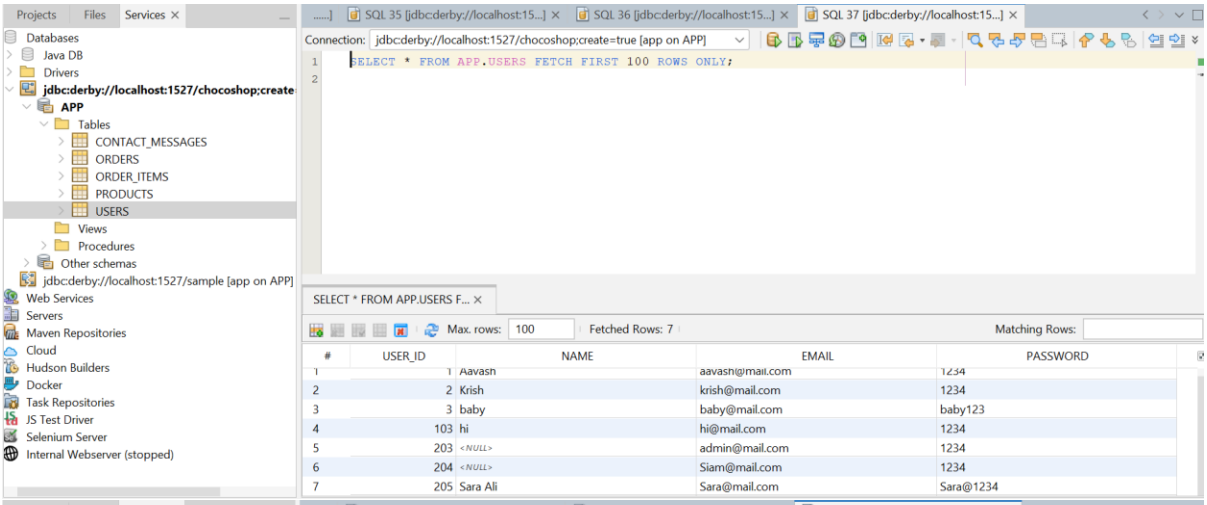
This Choco cart's UI follows Nielsen's 10 core principles. This core helps us to clean, intuitive shopping experience. (Nielsen, 1994)

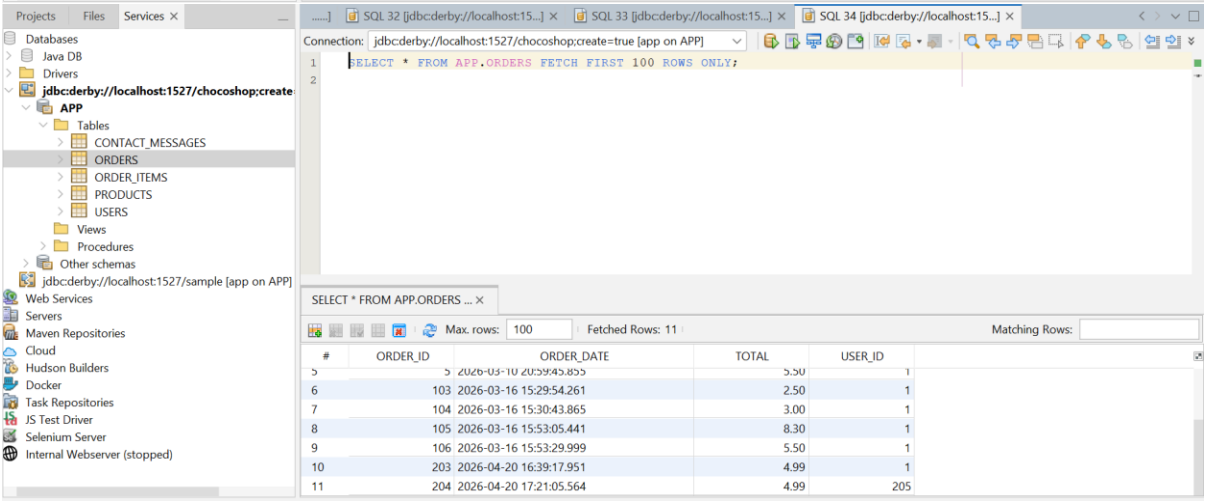
| Principle | Implementation in Choco cart |
|---|---|
| Visibility of system status | Ajax basket updates instantly, username display after login, order confirmation page shown post-checkout. |
| Consistency and standard | Uniform navigation bars and themes across all pages reduce learning time for new users. |
| Feedback | Dynamic messages and sats for the CRUD operators and conformation of the order. |
| Error prevention | Input validation on password and card details. |
| Recognition over all recall | Product details information and basket always items name, quantities, and prices |
| User control | Users can browse and manage basket without logging in. Shop follows three section structures (user account, Public, shopping Pages). This style minimizes stress to the system and makes it easy for buyers to buy 1st time. |

## 4. DATABASE

### Sample of the database record for the order history, usernames and messages from the users.




Projects
Files
Services
Databases
Java DB
Drivers
jdbc:derby://localhost:1527/chocoshop;create
APP
Tables
CONTACT_MESSAGES
ORDERS
ORDER_ITEMS
PRODUCTS
USERS
Views
Procedures
Other schemas
jdbc:derby://localhost:1527/sample [app on APP]
Web Services
Servers
Maven Repositories
Cloud
Hudson Builders
Docker
Task Repositories
JS Test Driver
Selenium Server
Internal Webserver (stopped)
SQL 35 [jdbc:derby://localhost:15...]
SQL 36 [jdbc:derby://localhost:15...]
SQL 37 [jdbc:derby://localhost:15...]
Connection: jdbc:derby://localhost:1527/chocoshop;create=true [app on APP]
SELECT * FROM APP.USERS FETCH FIRST 100 ROWS ONLY;
SELECT * FROM APP.USERS F...
Max. rows: 100
Fetched Rows: 7
Matching Rows:
# USER_ID NAME EMAIL PASSWORD
1 1 Aavash aavash@mail.com 1234
2 2 Krish krish@mail.com 1234
3 3 baby baby@mail.com baby123
4 103 hi hi@mail.com 1234
5 203 <NULL> admin@mail.com 1234
6 204 <NULL> Siam@mail.com 1234
7 205 Sara Ali Sara@mail.com Sara@1234


Projects
Files
Services
Databases
Java DB
Drivers
jdbc:derby://localhost:1527/chocoshop;create
APP
Tables
CONTACT_MESSAGES
ORDERS
ORDER_ITEMS
PRODUCTS
USERS
Views
Procedures
Other schemas
jdbc:derby://localhost:1527/sample [app on APP]
Web Services
Servers
Maven Repositories
Cloud
Hudson Builders
Docker
Task Repositories
JS Test Driver
Selenium Server
Internal Webserver (stopped)
SQL 32 [jdbc:derby://localhost:15...]
SQL 33 [jdbc:derby://localhost:15...]
SQL 34 [jdbc:derby://localhost:15...]
Connection: jdbc:derby://localhost:1527/chocoshop;create=true [app on APP]
SELECT * FROM APP.ORDERS FETCH FIRST 100 ROWS ONLY;
SELECT * FROM APP.ORDERS ...
Max. rows: 100
Fetched Rows: 11
Matching Rows:
# ORDER_ID ORDER_DATE TOTAL USER_ID
5 5 2026-03-10 20:59:45.855 5.50 1
6 103 2026-03-16 15:29:54.261 2.50 1
7 104 2026-03-16 15:30:43.865 3.00 1
8 105 2026-03-16 15:53:05.441 8.30 1
9 106 2026-03-16 15:53:29.999 5.50 1
10 203 2026-04-20 16:39:17.951 4.99 1
11 204 2026-04-20 17:21:05.564 4.99 205

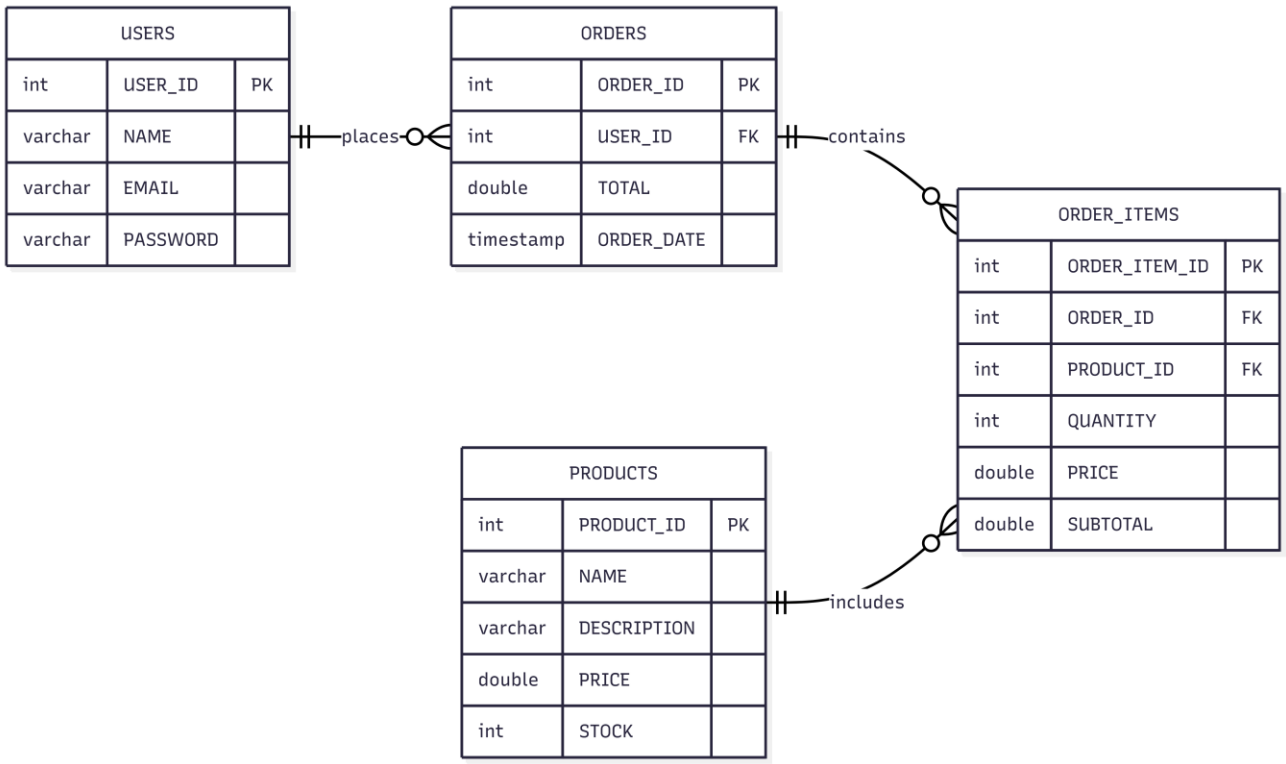



Fig. 4.1. ER diagram

| Table | Main fields | Purpose |
|---|---|---|
| USERS | USER_ID, NAME, EMAIL, PASSWORD | Stores registered user details for login and account creation. |
| PRODUCTS | PRODUCT_ID, NAME, DESCRIPTION, PRICE, STOCK | Stores product details shown on the products page, including available stock. |
| ORDERS | ORDER_ID, USER_ID, TOTAL, ORDER_DATE | Stores each completed order and links it to the user who placed it. |
| ORDER_ITEMS | ORDER_ITEM_ID, ORDER_ID, PRODUCT_ID, QUANTITY, PRICE, SUBTOTAL | Stores the individual products that belong to each order. |

## 4.1. The database includes a suitable mix of field types. For example:

Integer: USER_ID, PRODUCT_ID, ORDER_ID, STOCK, QUANTITY

Text / Varchar: NAME, EMAIL, PASSWORD, DESCRIPTION

Decimal / Double: PRICE, TOTAL, SUBTOTAL

Timestamp / Date-Time: ORDER_DATE

## 4.2. Relationship table

| Table 1 | Table 2 | Relationship |
|---|---|---|
| USERS | ORDERS | One-to-many |
| ORDERS | ORDER_ITEMS | One-to-many |
| PRODUCTS | ORDER_ITEMS | One-to-many |

# 5. TESTING AND IMPLEMENTATION:

## 5.1 Menu Structure / Navigation

Choco Cart implements a consistent navigation bar on every page using JSF component tags. The navbar renders conditionally based on the user's session state: Login and Register links appear for guests, while a personalised Welcome message, Logout button, and (for admin) Admin link appear for authenticated users.

**How it works:**

The navbar is built using <h:link> tags linked to JSF outcomes (page names). Conditional rendering uses rendered="#{loginBean.loggedIn}" and rendered="#{!loginBean.loggedIn}" to show/hide appropriate links. The admin link uses rendered="#{loginBean.admin}".

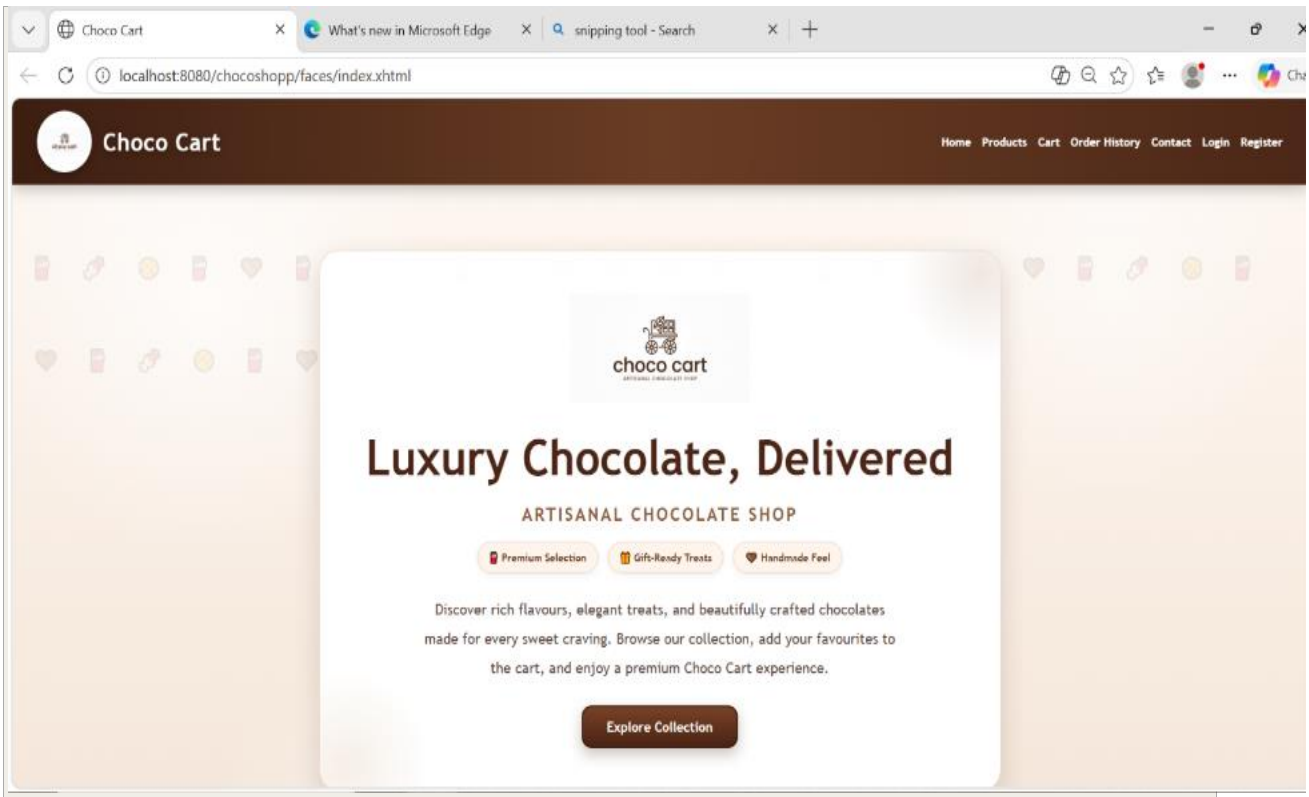



*Fig 5.1.1 – Home page showing full navigation bar (guest view: Login and Register visible)*

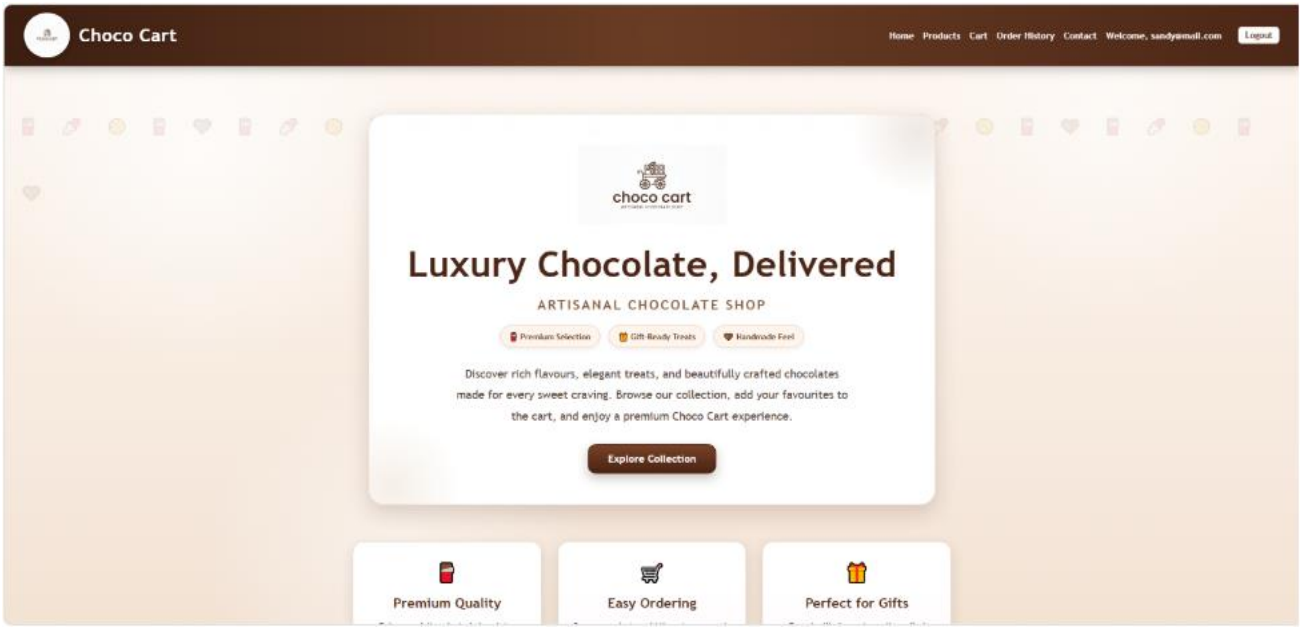



*Fig 5.1.2 – Products page showing navigation bar after login (Welcome message + Logout visible, Login/Register hidden)*

*The screenshots above confirm that the navigation bar is present and functional across multiple pages, with dynamic link rendering based on session state.*

## 5.2 CRUD Operations

### 5.2.1 Create

The Create operation is demonstrated through three distinct user actions: user registration, order placement, and contact form submission. Each action inserts new records into the corresponding database table.

**(a) User Registration – INSERT into USERS table**

When a new user completes the registration form and submits it, RegisterBean.register() is called. After passing all validation checks, it invokes UserDAO.registerUser() which executes an SQL INSERT statement to create the new account.

**RegisterBean.java – register() method (key validation and INSERT logic):**

```java
@Named("registerBean")
@RequestScoped
public class RegisterBean implements Serializable {

    private String name;
    private String email;
    private String password;
    private String confirmPassword;
    private String message;

    public RegisterBean() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
```

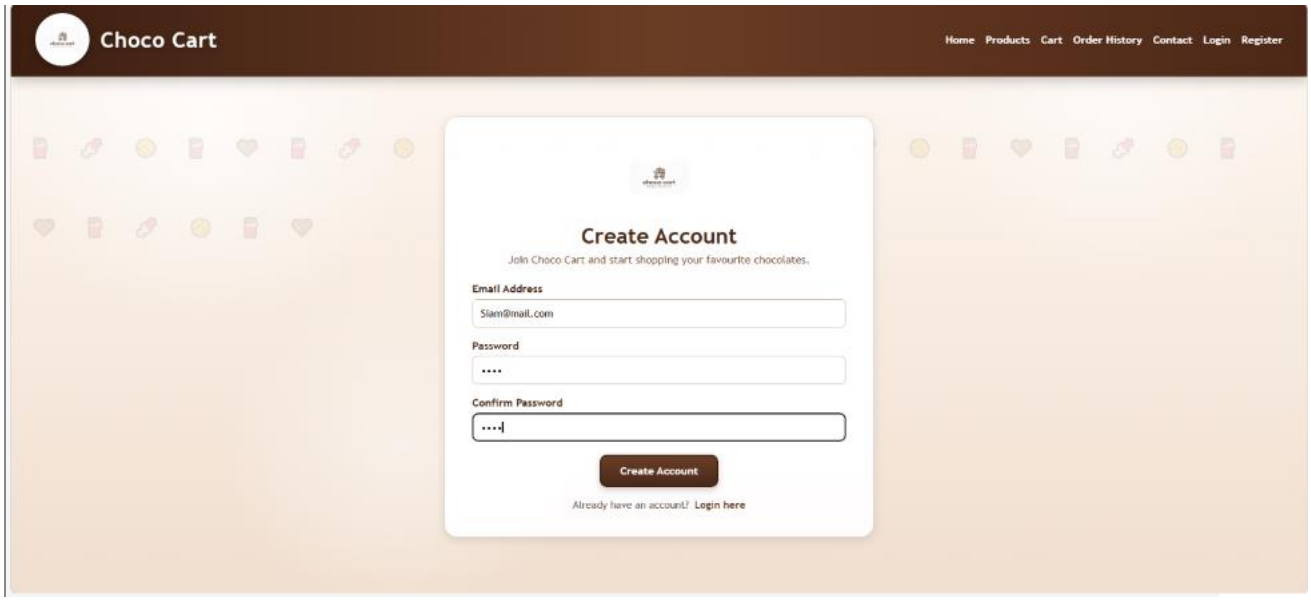



*Fig 5.2.1a – Registration form with test data entered (Name, Email, Password, Confirm Password)*

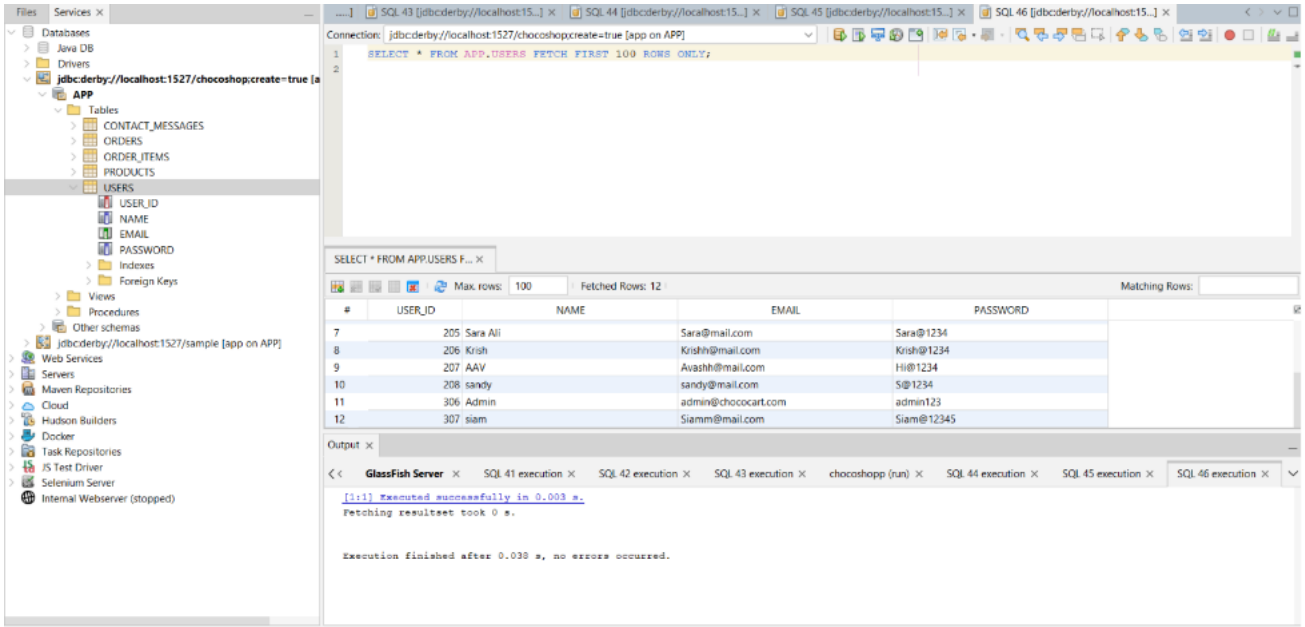



*Fig 5.2.1b – USERS database table after submission, showing new row inserted with the test email*

### (b) Order Placement – INSERT into ORDERS and ORDER_ITEMS tables

When a user completes payment, PaymentBean.processPayment() saves the order to the database via OrderDAO.addOrder(). This inserts one row into ORDERS and one row per cart item into ORDER_ITEMS.

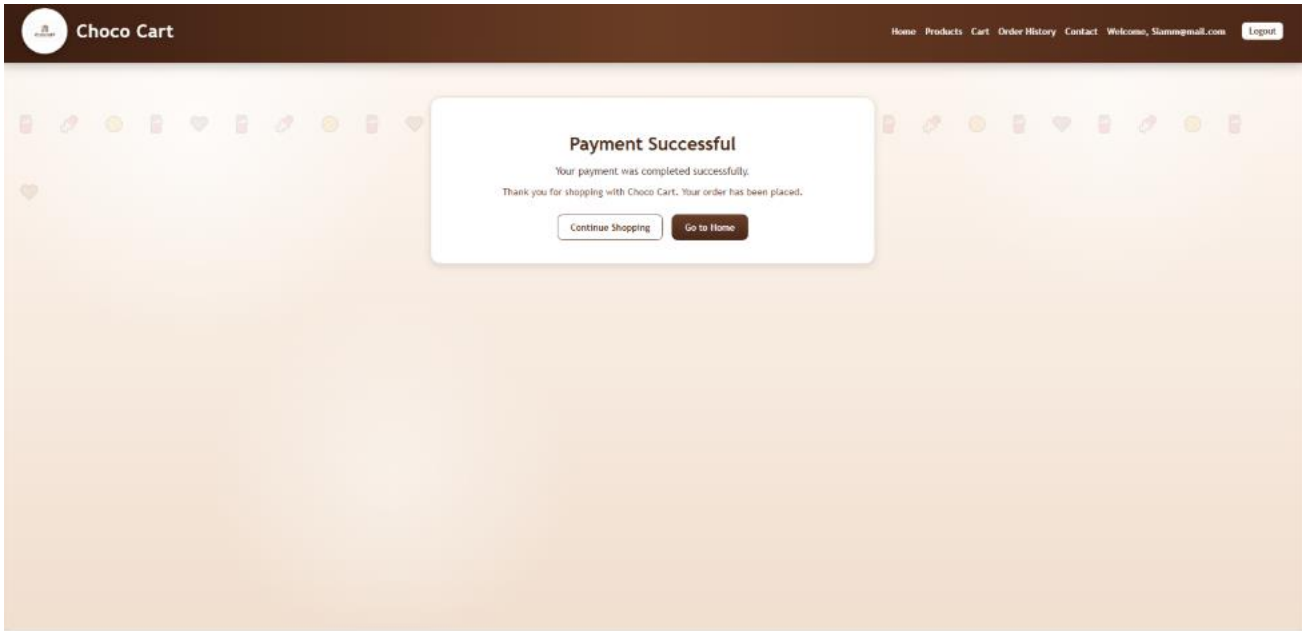



*Fig 5.2.1c – Payment Success page confirming order was placed*

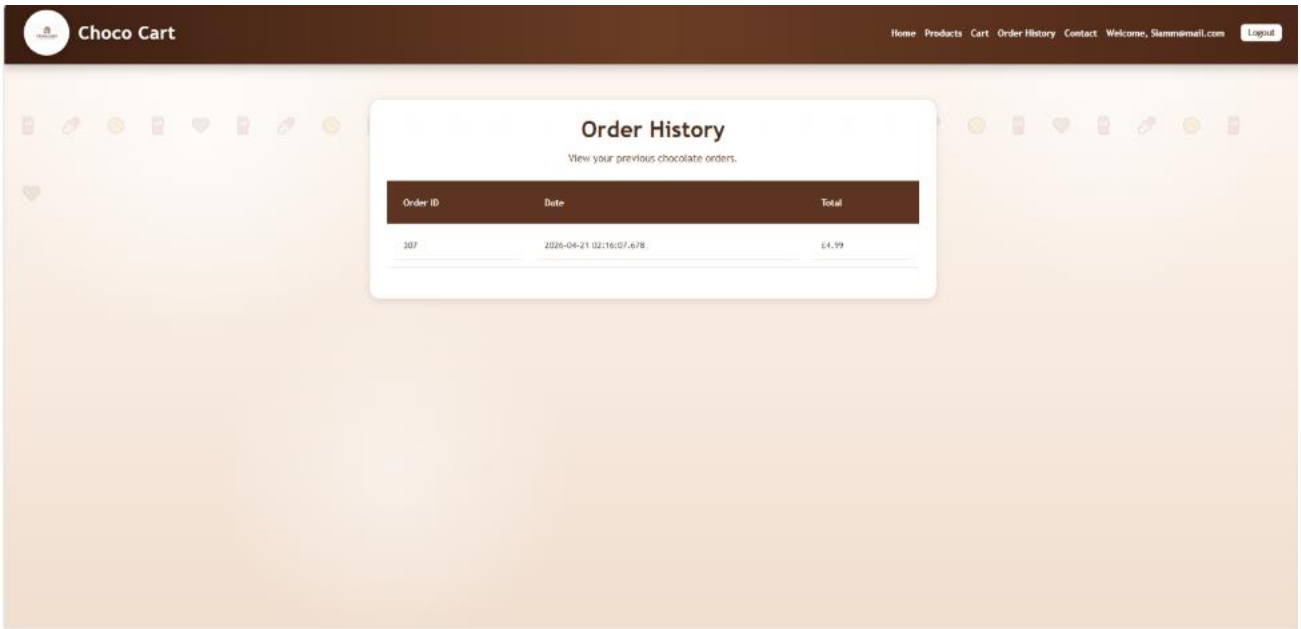



*Fig 5.2.1d – ORDERS table showing new order row with USER_ID, TOTAL, and ORDER_DATE*

The Contact Us page allows any user (guest or logged-in) to submit an enquiry. ContactBean.submit() calls ContactDAO.saveMessage() which inserts the message into the CONTACT_MESSAGES table.

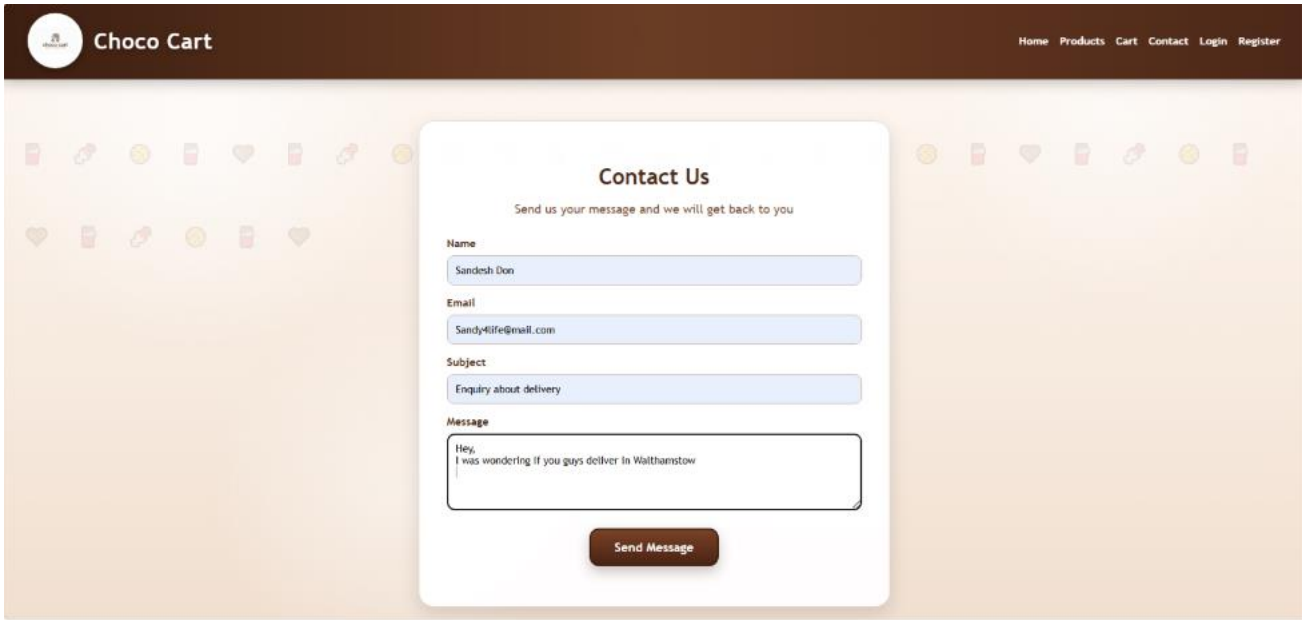



*Fig 5.2.1f – Contact Us form filled with test name, email, and message*

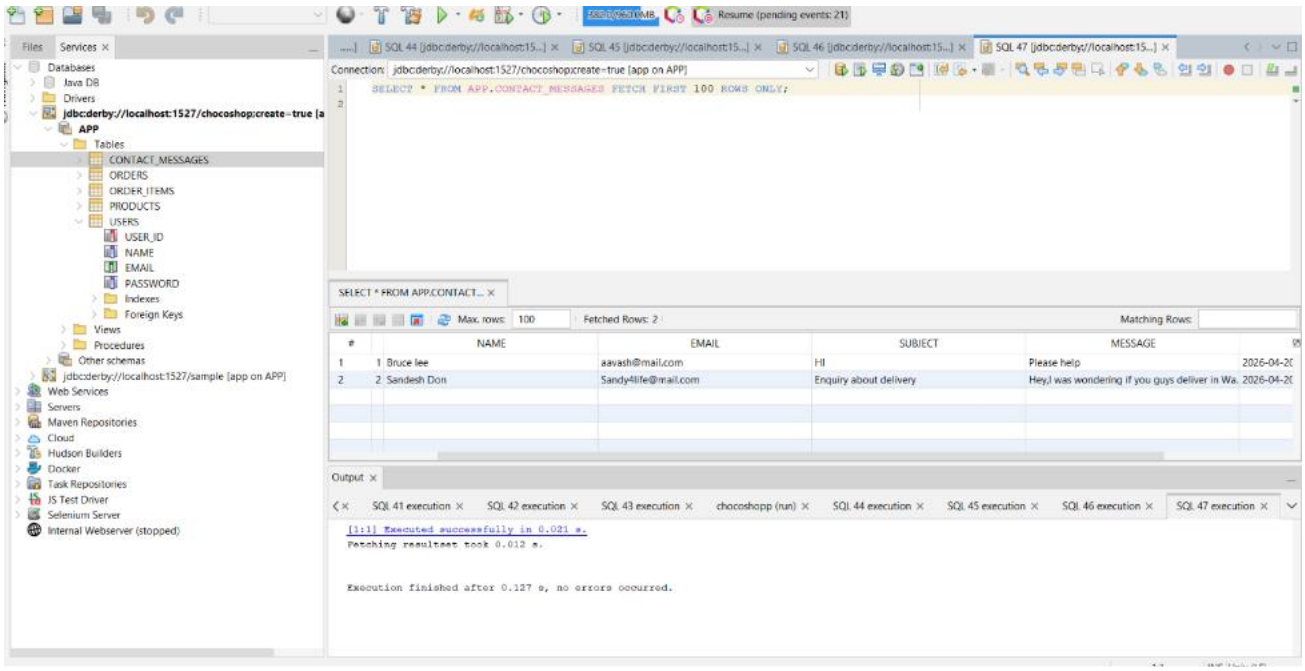



*Fig 5.2.1g – CONTACT_MESSAGES table showing the new row after submission*

### 5.2.2 Read

The Read operation retrieves stored data from the database and displays it to the user. Two main Read scenarios are implemented: the product catalogue and the admin order view.

### (a) Product Catalogue – SELECT from PRODUCTS table

On page load, ProductBean (a @RequestScoped managed bean) calls ProductDAO.getAllProducts() which executes SELECT * FROM PRODUCTS and returns the results as a List<Product>. The JSF products.xhtml page iterates over this list using <ui:repeat> to render each product card dynamically.

**ProductDAO.java – getAllProducts() method:**

```java
import java.util.List;

public class ProductDAO {

    public List<Product> getAllProducts() {
        List<Product> products = new ArrayList<>();

        try {
            Connection conn = DBConnection.getConnection();
            System.out.println("Connected to DB");

            String sql = "SELECT * FROM PRODUCTS";
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                Product p = new Product();
                p.setProductId(rs.getInt("PRODUCT_ID"));
                p.setName(rs.getString("NAME"));
                p.setDescription(rs.getString("DESCRIPTION"));
                p.setPrice(rs.getDouble("PRICE"));
                p.setStock(rs.getInt("STOCK"));
                products.add(p);
            }

            System.out.println("Products loaded: " + products.size());

            rs.close();
            ps.close();
            conn.close();

        } catch (Exception e) {
            e.printStackTrace();
        }

        return products;
    }
```

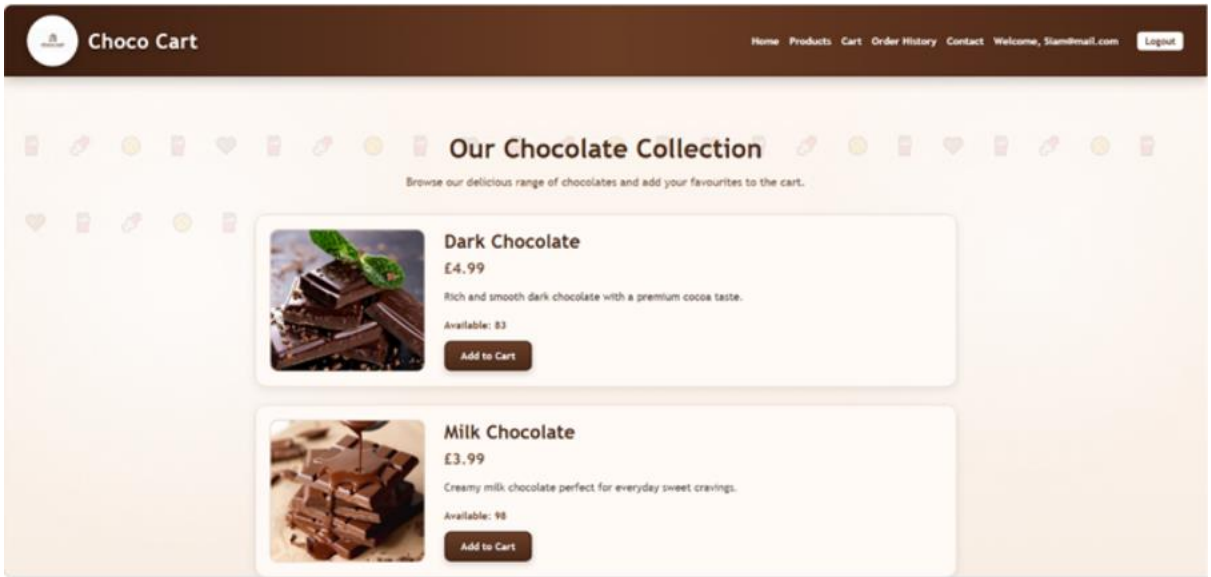



*Fig 5.2.2a – Products page showing chocolate cards loaded from the PRODUCTS database table*

### (b) Admin Order View – SELECT from ORDERS joined with USERS

When an admin logs in, the admin.xhtml page loads all orders via AdminBean, calling OrderDAO which executes a JOIN query across ORDERS and USERS to display order ID, user name, date, and total.

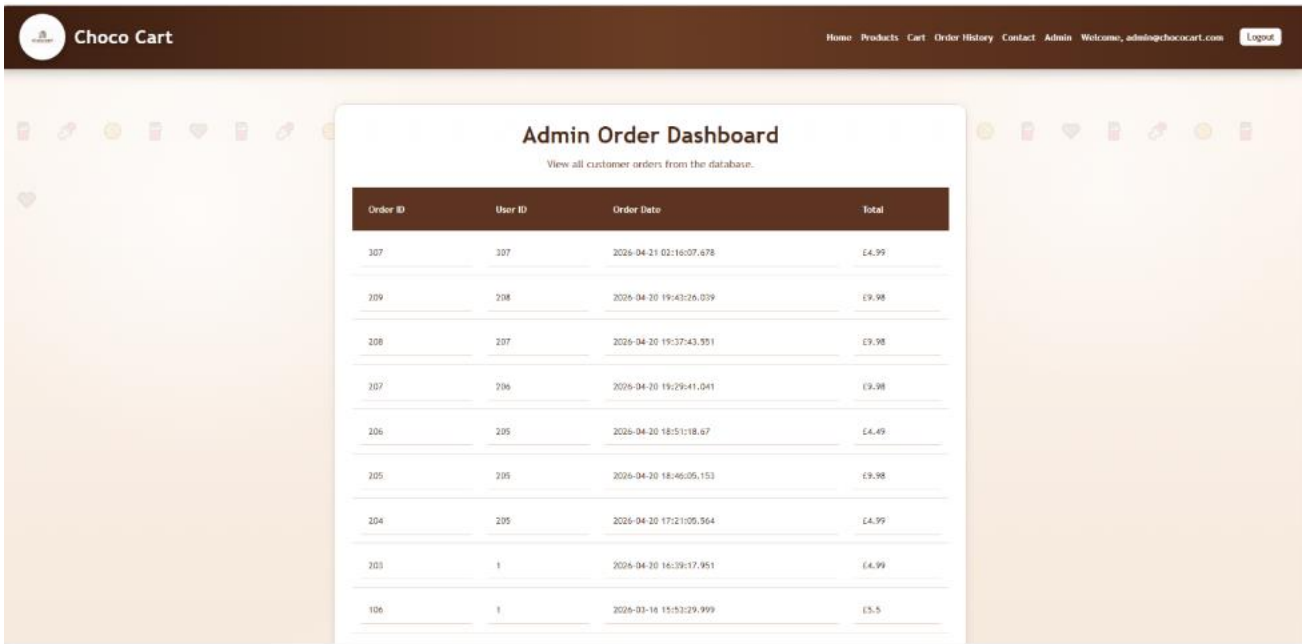



*Fig 5.2.2b – Admin page displaying all customer orders retrieved from the database*

### 5.2.3 Update

The Update operation is demonstrated through two mechanisms: quantity adjustment in the cart, and stock reduction after a successful purchase.

**(a) Cart Quantity Update (Ajax-powered, in-memory)**

When the user changes the quantity number in the cart table and presses a key, a JSF Ajax event fires and calls cartBean.updateQuantity(item) which adjusts the CartItem's quantity and triggers a recalculation of the subtotal and total via Ajax render (no page reload).

**cart.xhtml – quantity input with f:ajax:**

```
<h:inputText value="#{item.quantity}"
             style="width:60px; text-align:center;">
    <f:ajax event="keyup"
            listener="#{cartBean.updateQuantity(item)}"
            render="cartTable totalPanel cartMessage"/>
</h:inputText>
```

**CartBean.java – updateQuantity() method:**

```
public void updateQuantity(CartItem item) {
    // Prevent quantity dropping below 1
    if (item.getQuantity() < 1) {
        item.setQuantity(1);
    }
    // Subtotal recalculates automatically via CartItem.getSubtotal()
    // Ajax render="cartTable totalPanel" updates the UI without page reload
}
```

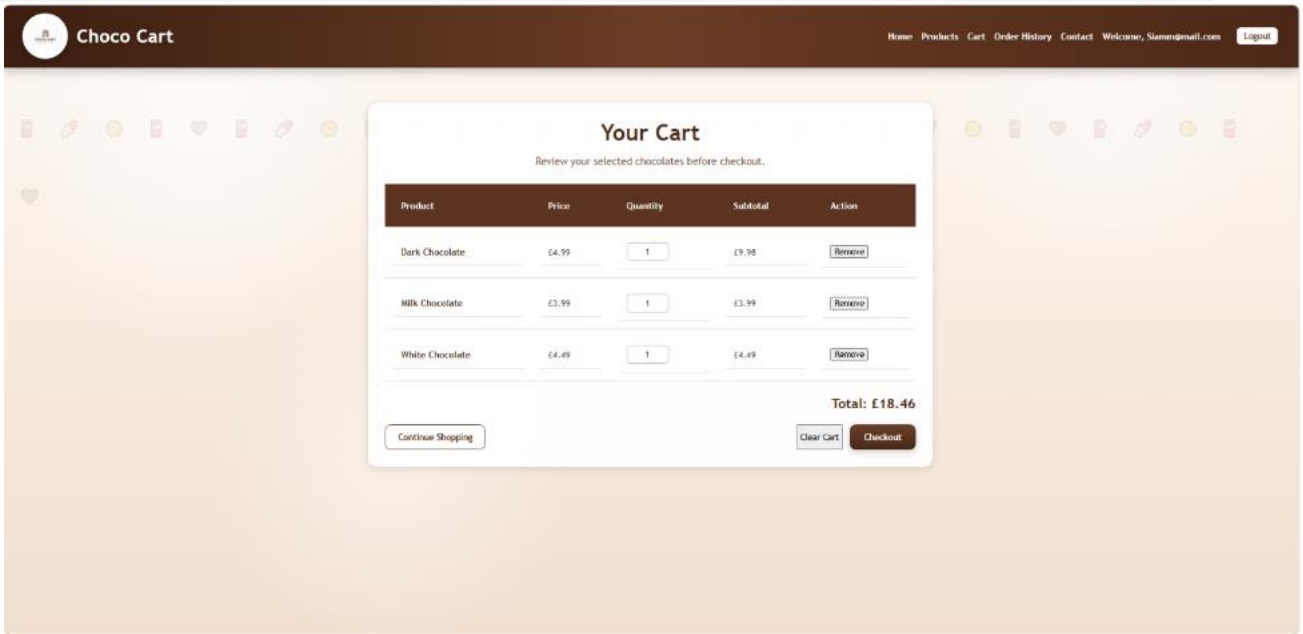



*Fig 5.2.3a – Cart page before quantity change (qty=1, subtotal shown)*

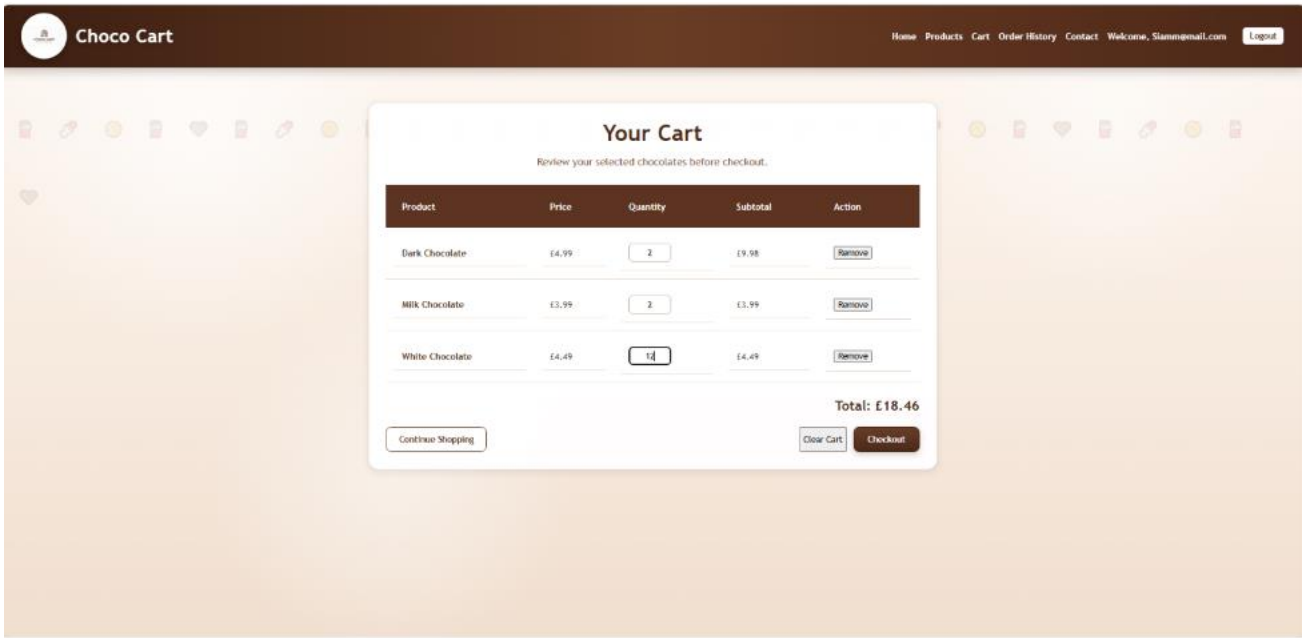



*Fig 5.2.3b – Cart page after quantity changed to 2 via keyup – subtotal and total update instantly (Ajax, no reload)*

**(b) Stock Reduction – UPDATE PRODUCTS table on payment**

After a successful payment, PaymentBean calls ProductDAO.reduceStock() for each cart item. This executes an UPDATE SQL statement to decrement STOCK by the purchased quantity.

**ProductDAO.java – reduceStock() method:**

```java
    public void reduceStock(String productName, int quantityBought) {
        try {
            Connection conn = DBConnection.getConnection();

            String sql = "UPDATE PRODUCTS SET STOCK = STOCK - ? WHERE NAME = ? AND STOCK >= ?";
            PreparedStatement ps = conn.prepareStatement(sql);
            ps.setInt(1, quantityBought);
            ps.setString(2, productName);
            ps.setInt(3, quantityBought);

            ps.executeUpdate();

            ps.close();
            conn.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

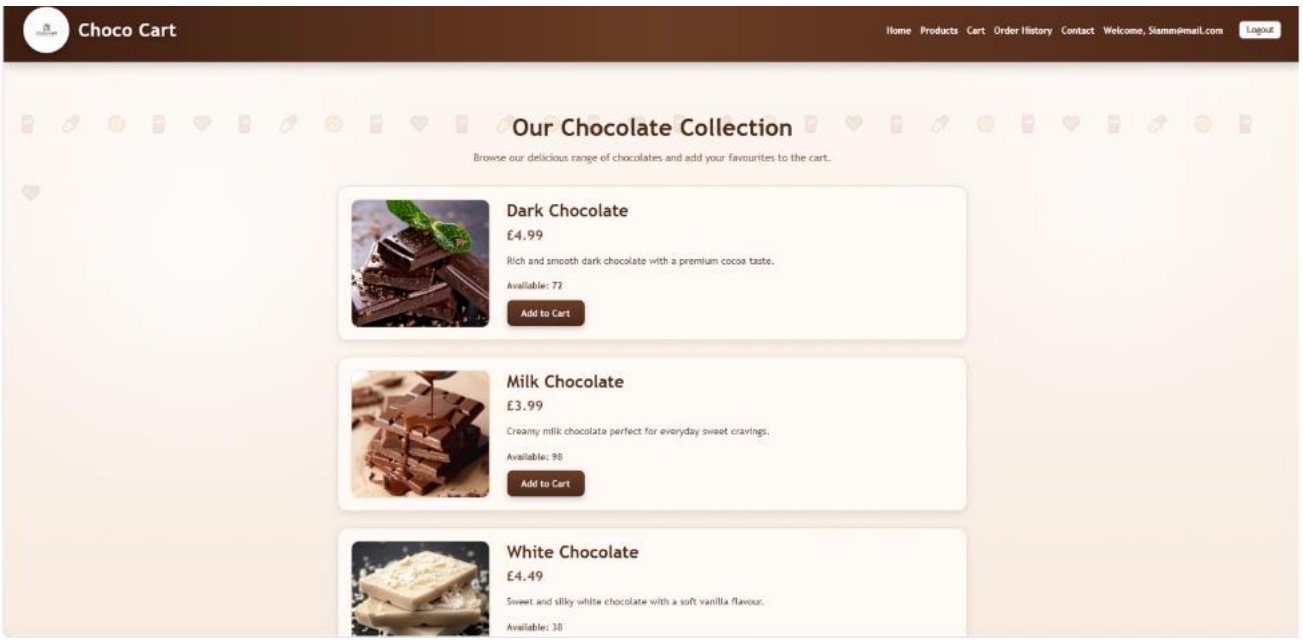



*Fig 5.2.3c – PRODUCTS table before purchase (e.g. Dark Chocolate STOCK = 72)*

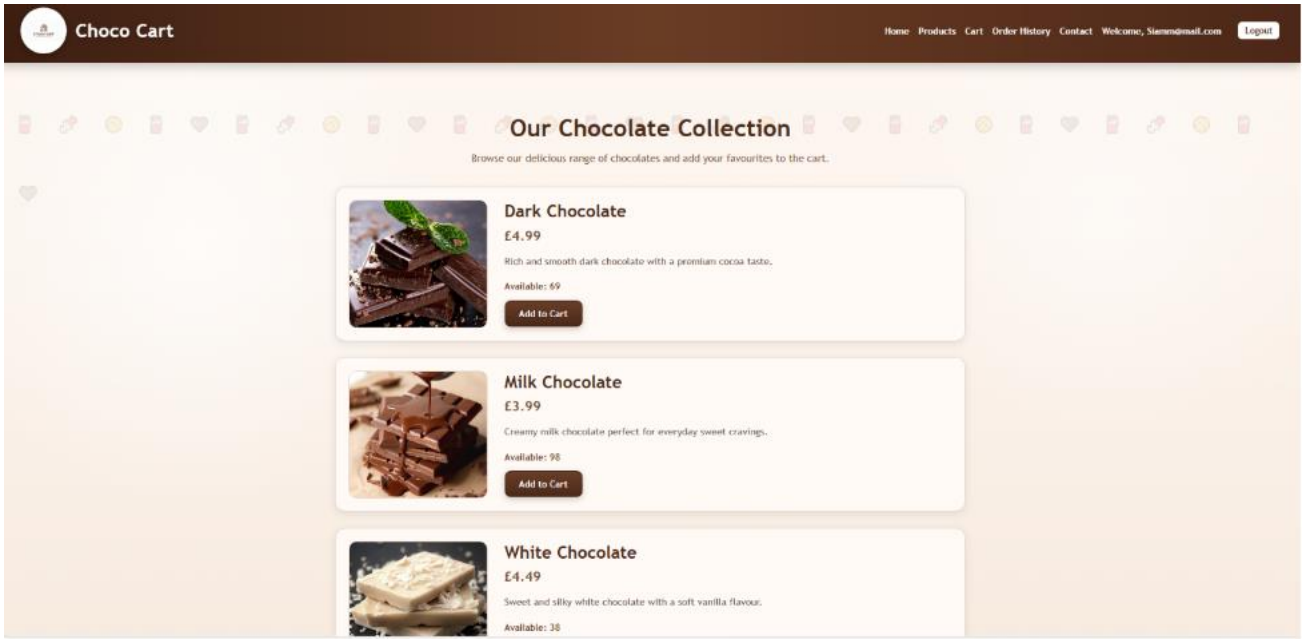



*Fig 5.2.3d – PRODUCTS table after purchase of 2 units (STOCK = 69, reduced by PaymentBean)*

### 5.2.4 Delete

The Delete operation is demonstrated through cart item removal. When the user clicks the Remove button on a cart row, cartBean.removeItem(item) is called. Ajax immediately re-renders the cart table, total panel, and message area without a page reload. If the last item is removed, an empty cart message appears.

**cart.xhtml – Remove button with f:ajax:**

```
<h:commandButton value="Remove"
                 action="#{cartBean.removeItem(item)}">
    <f:ajax execute="@this"
            render="cartTable totalPanel cartMessage"/>
</h:commandButton>
```

**CartBean.java – removeItem() method:**

```
public void removeItem(CartItem item) {
    cartItems.remove(item);
    // Ajax re-renders cartTable and totalPanel automatically
    // If cartItems is now empty, the empty-cart panel renders instead
}
```

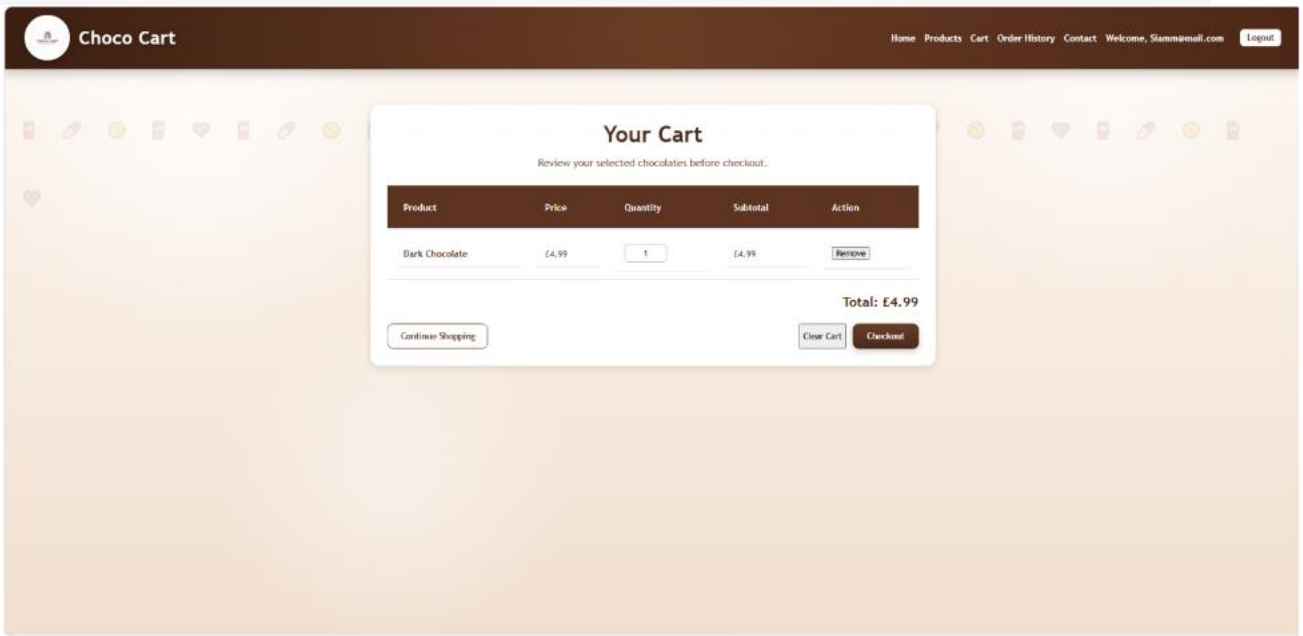



*Fig 5.2.4a – Cart page with one item present (before delete)*

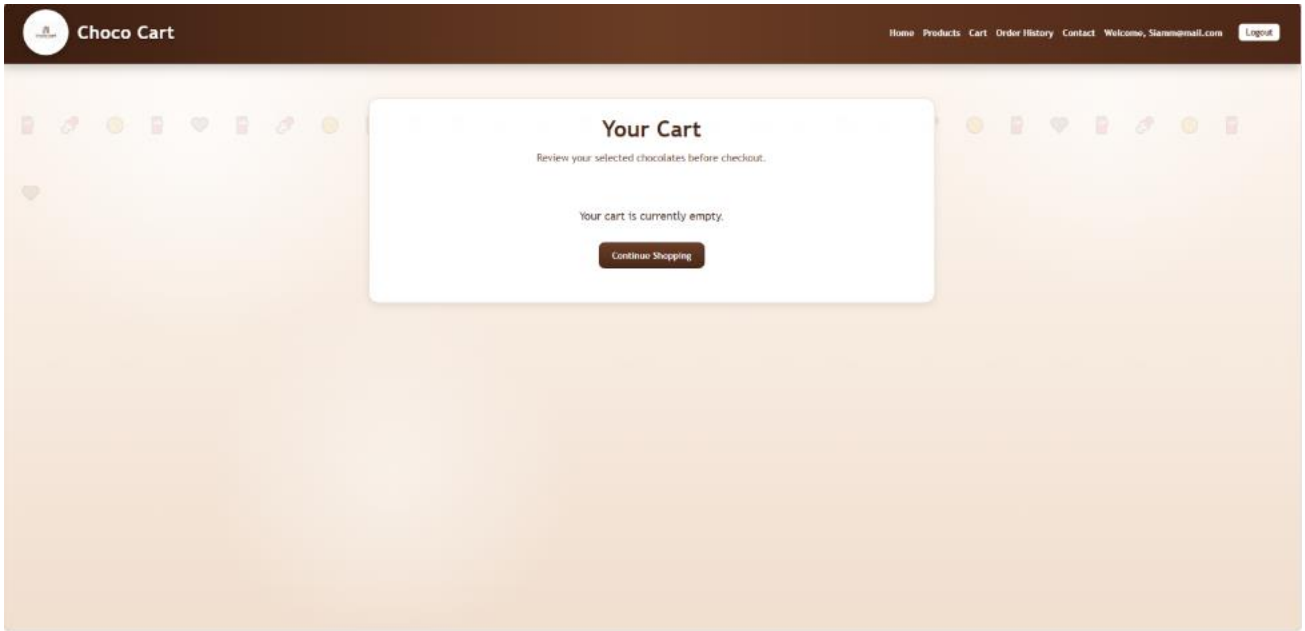



*Fig 5.2.4b – Cart page immediately after clicking Remove – item gone, total updated to £0.00, empty message shown (Ajax, no reload)*

## 5.3 Validation

Choco Cart implements server-side validation in two areas: user registration and payment checkout.

### 5.3.1 Registration Validation

RegisterBean.register() applies the following checks in sequence before writing to the database:

- All four fields (name, email, password, confirmPassword) must be non-null and non-empty.
- The email must not already exist in the USERS table (UserDAO.emailExists() check).
- The password and confirm password values must match exactly.
- If any check fails, an error message is set and the form re-renders with no database write.

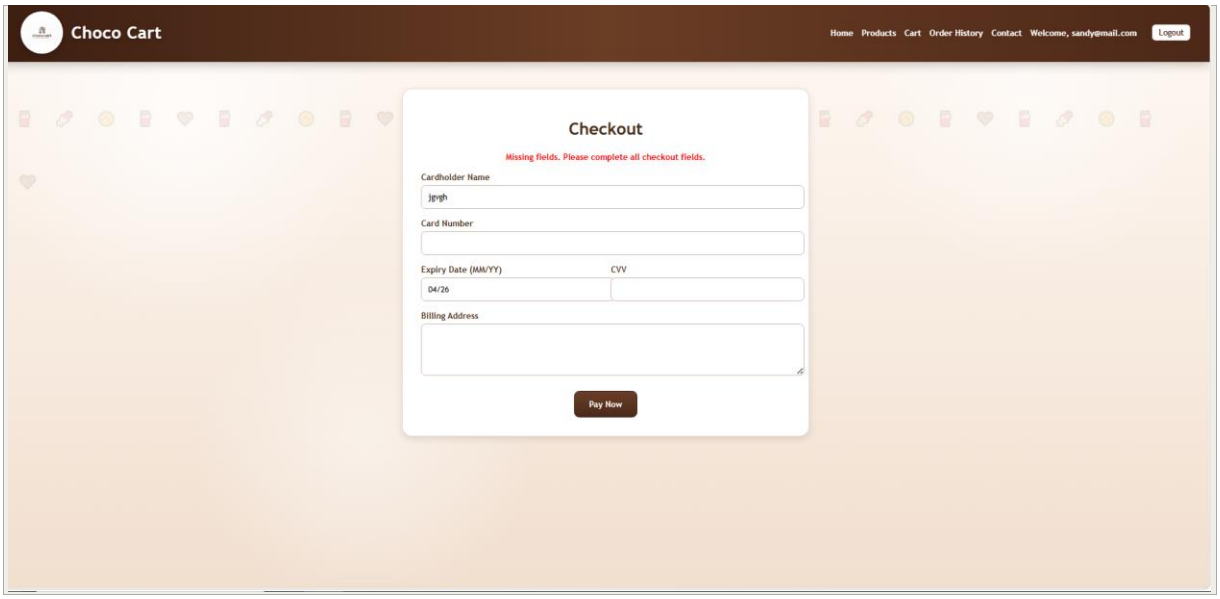



*Fig 5.3.1a – Registration form submitted with all fields empty – error: 'Missing fields. Please fill in all fields.'*

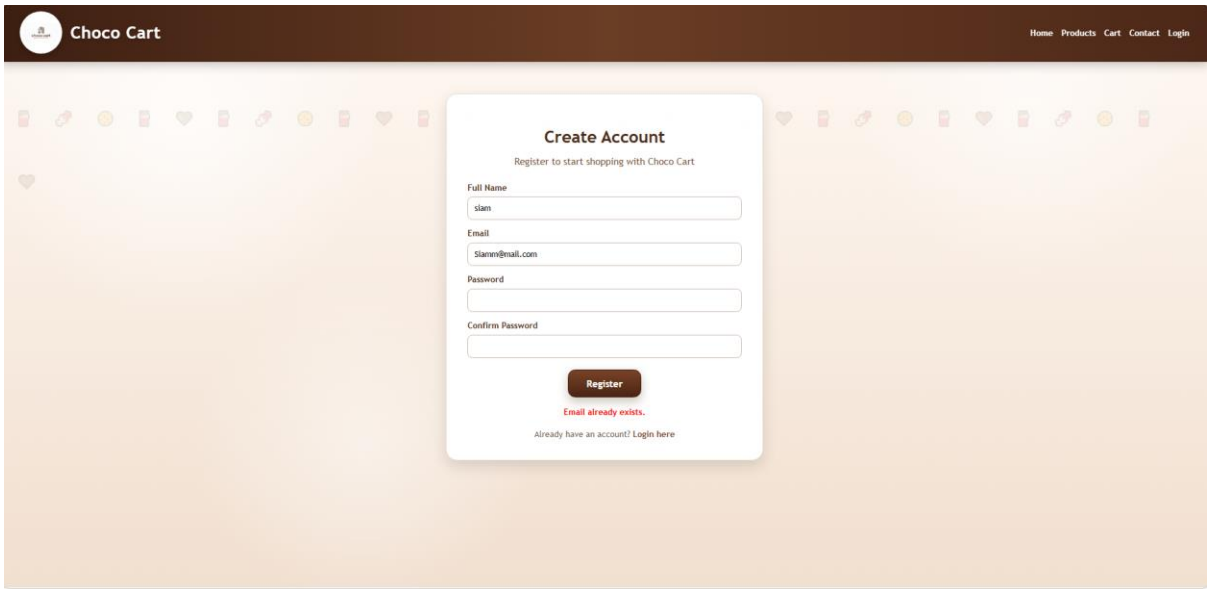



*Fig 5.3.1b – Registration form submitted with a duplicate email – error: 'Email already exists.'*

### 5.3.2 Payment / Checkout Validation

PaymentBean.processPayment() applies four regex-based validations on the payment form before any database writes occur:

**PaymentBean.java – validation code:**

```java
            message = "Your cart is empty.";
            return null;
        }

        if (loginBean == null || !loginBean.isLoggedIn()) {
            message = "Please login before checkout.";
            return "login?faces-redirect=true";
        }

        if (cardholderName == null || cardholderName.trim().isEmpty()
                || cardNumber == null || cardNumber.trim().isEmpty()
                || expiryDate == null || expiryDate.trim().isEmpty()
                || cvv == null || cvv.trim().isEmpty()
                || billingAddress == null || billingAddress.trim().isEmpty()) {
            message = "Missing fields. Please complete all checkout fields.";
            return null;
        }

        String cleanedCardNumber = cardNumber.replaceAll("\\s+", "");

        if (!cleanedCardNumber.matches("\\d{16}")) {
            message = "Card number must be exactly 16 digits.";
            return null;
        }

        if (!cvv.matches("\\d{3}")) {
            message = "CVV must be exactly 3 digits.";
            return null;
        }

        if (!expiryDate.matches("(0[1-9]|1[0-2])/\\d{2}")) {
            message = "Expiry date must be in MM/YY format.";
            return null;
        }

        ProductDAO productDAO = new ProductDAO();
        OrderDAO orderDAO = new OrderDAO();
```

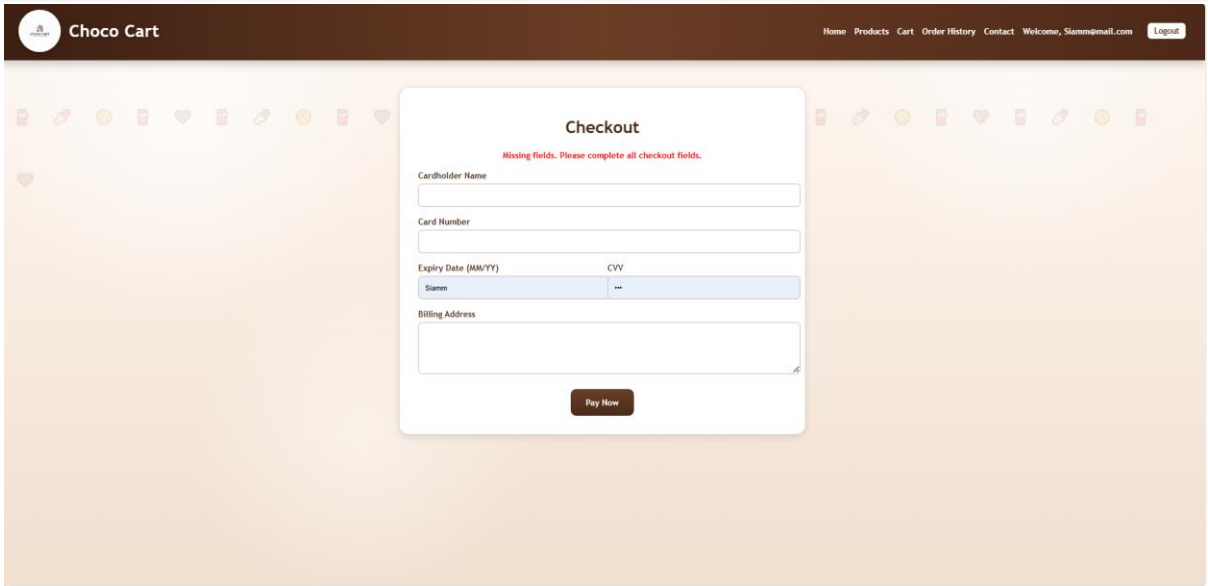



*Fig 5.3.2a – Checkout form submitted with empty fields – error message displayed on the page*

## 5.4 Ajax

Ajax is implemented using JSF's built-in <f:ajax> tag. It enables partial page updates (only specific components are refreshed) without a full browser reload, providing a fast, responsive user experience. Three Ajax scenarios are implemented:

### 5.4.1 Cart Quantity Update (keyup event)

When the user types a new quantity in the cart table, a keyup event fires, calling updateQuantity() on the server. The response updates only the cartTable, totalPanel, and cartMessage components.

**Code (cart.xhtml):**

```
<h:inputText value="#{item.quantity}">
    <f:ajax event="keyup"
            listener="#{cartBean.updateQuantity(item)}"
            render="cartTable totalPanel cartMessage"/>
</h:inputText>
```

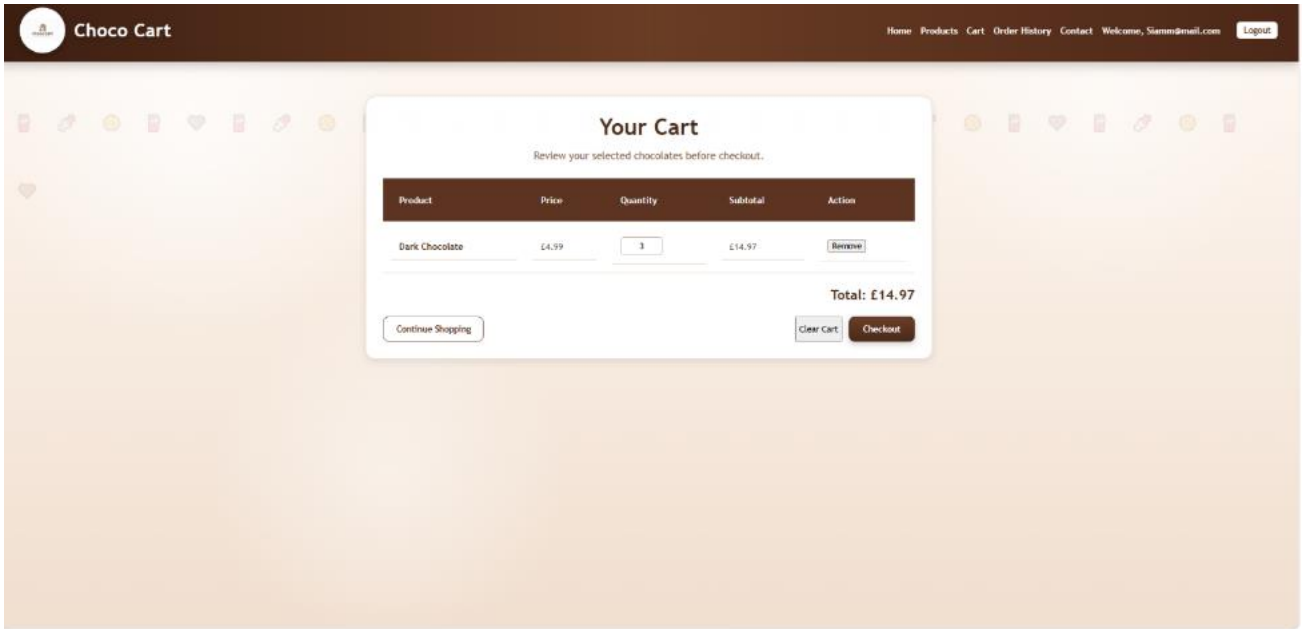


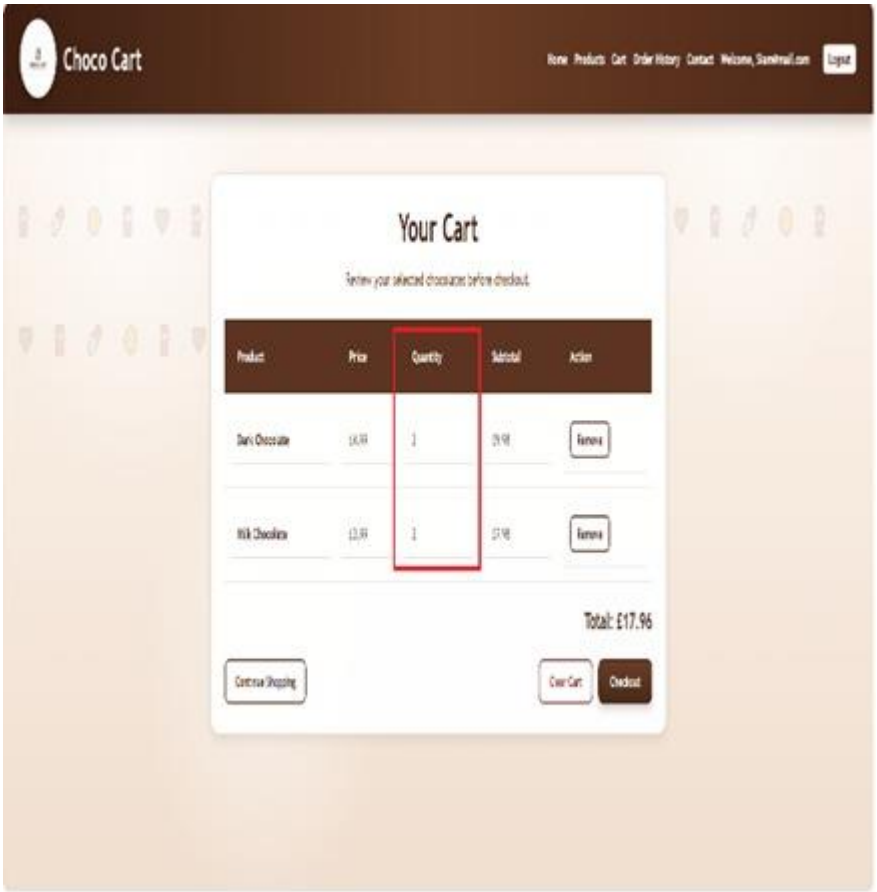

*Fig 5.4.1 – Cart with quantity changed to 3 – subtotal and total updated via Ajax (no page reload; network tab shows partial response)*

## 5.4.2 Cart Item Remove (commandButton with Ajax)

The Remove button uses <f:ajax execute="@this"> so only the button action is submitted to the server. The response re-renders the table and total, giving an instant removal effect.

**Code (cart.xhtml):**

```
<h:commandButton value="Remove"
                 action="#{cartBean.removeItem(item)}">
    <f:ajax execute="@this"
            render="cartTable totalPanel cartMessage"/>
</h:commandButton>
```



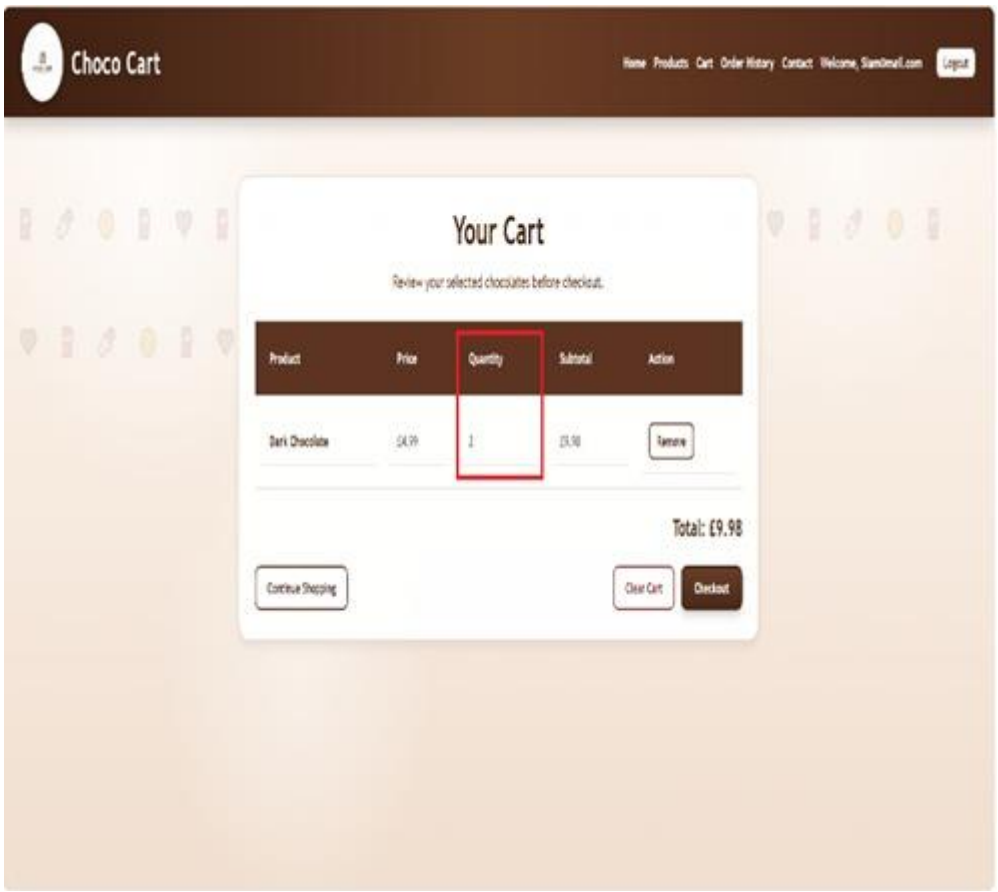




*Fig 5.4.2 – Remove button clicked – item disappears and total recalculates without page reload*

## 5.4.3 Clear Cart (commandButton with Ajax)

The Clear Cart button also uses f:ajax, calling cartBean.clearCart(). After the server processes the call, the same cartTable and totalPanel components are re-rendered, now reflecting an empty cart.

```
<h:commandButton value="Clear Cart"
                 action="#{cartBean.clearCart}">
    <f:ajax execute="@this"
            render="cartTable totalPanel cartMessage"/>
</h:commandButton>
```

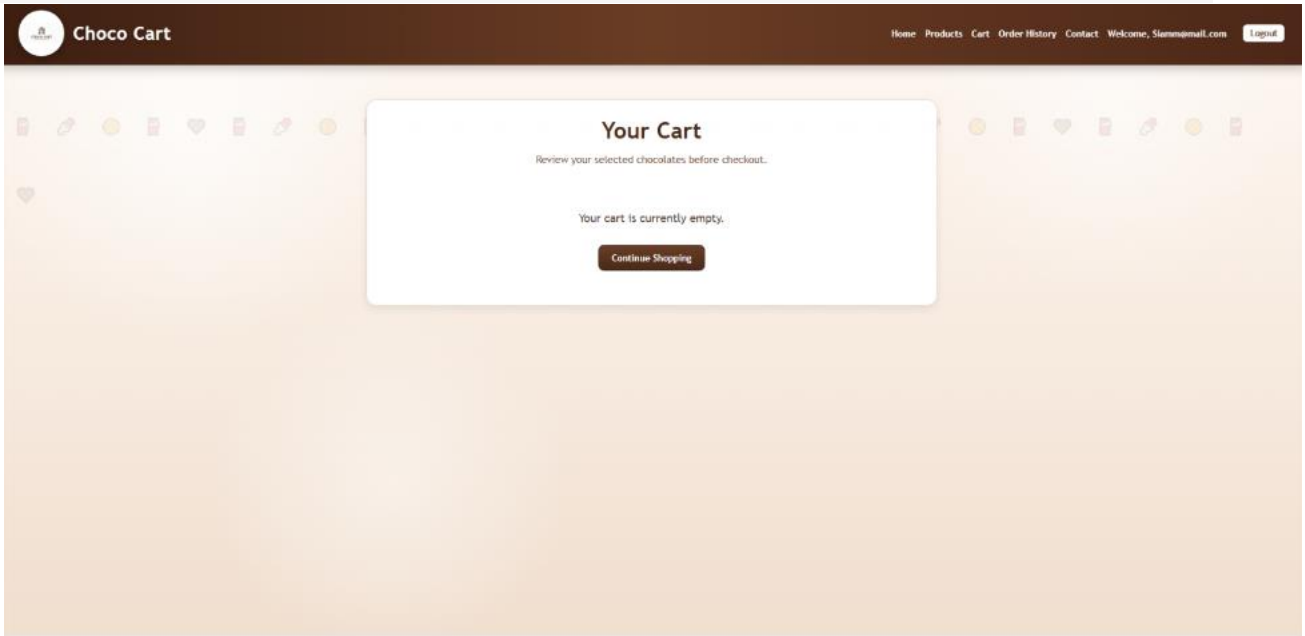



*Fig 5.4.3 – Clear Cart clicked – all items removed and empty cart message shown via Ajax*

## 5.5 Session Tracking

Choco Cart uses two @SessionScoped managed beans to maintain state across all pages of the application:

- LoginBean (@SessionScoped): stores the logged-in user's email, name, userId, and admin flag for the duration of the browser session.
- CartBean (@SessionScoped): stores the list of CartItem objects. Guest users can add to the cart; the basket persists even after login, so items are not lost during the authentication redirect.

**LoginBean.java – session scope declaration:**

```java
import model.User;

@Named("loginBean")
@SessionScoped
public class LoginBean implements Serializable {

    private String email;
    private String password;
    private String name;
    private int userId;
    private boolean loggedIn = false;
    private boolean admin = false;
    private String message;

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getName() {
        return name;
    }

    public int getUserId() {
        return userId;
    }
```

**How session tracking is visible to the user:**

- After login, the navbar on every page displays: 'Welcome, [username]' and a Logout button.
- The cart contents (CartBean) are preserved when navigating between Products, Cart, and Checkout.
- If a logged-in user navigates back to the login page, they are not forced to re-authenticate.
- On logout, LoginBean fields are nulled and the session effectively resets.

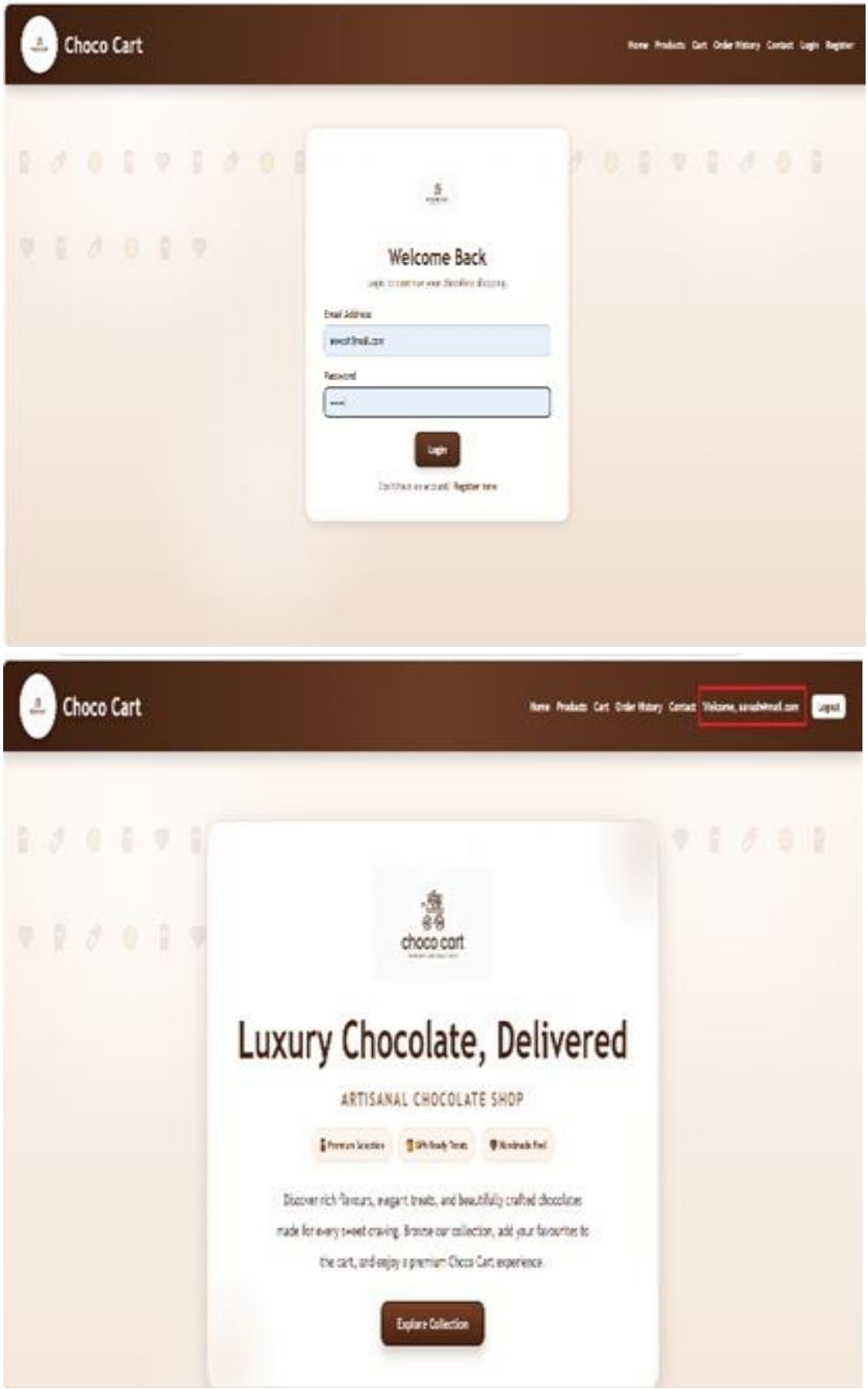
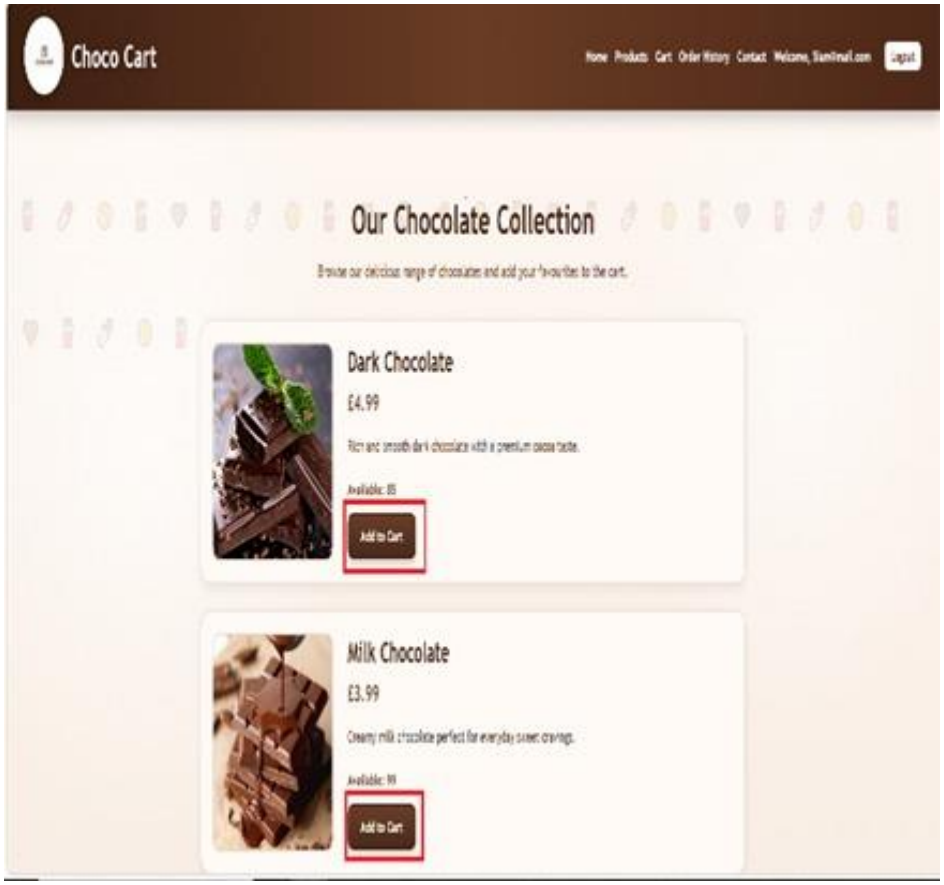



*Fig 5.5.1 – Home page after login showing 'Welcome, test@test.com' in navbar (session data visible)*



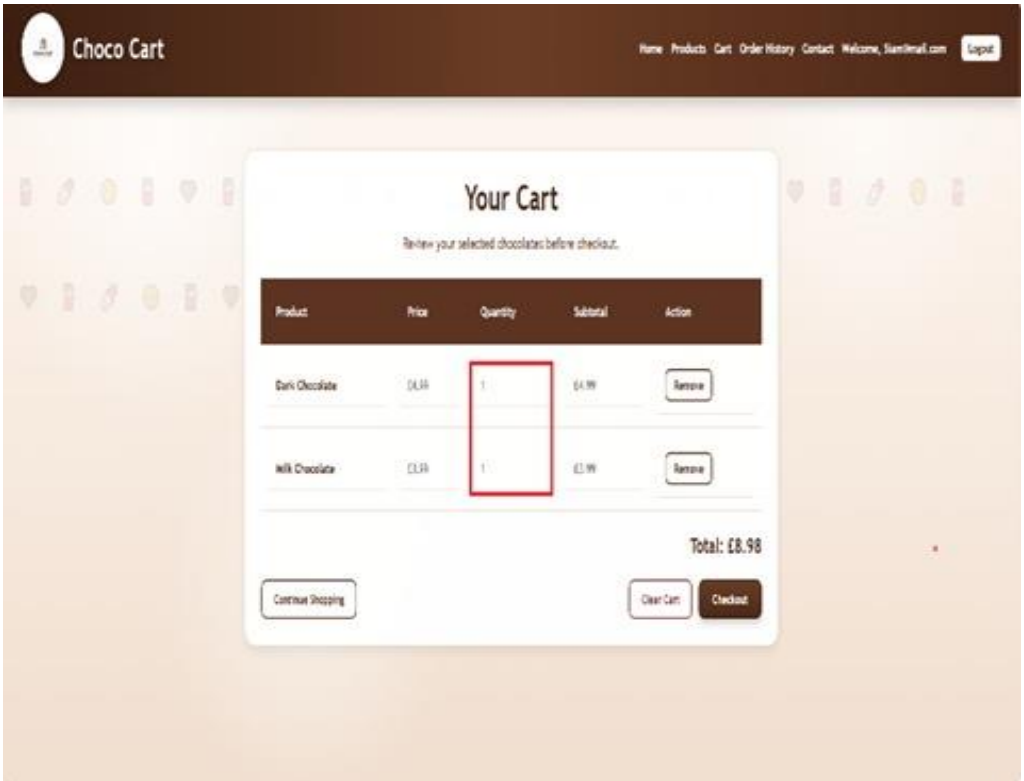




*Fig 5.5.2 – Products page showing same 'Welcome' message – confirms session persists across page navigation*

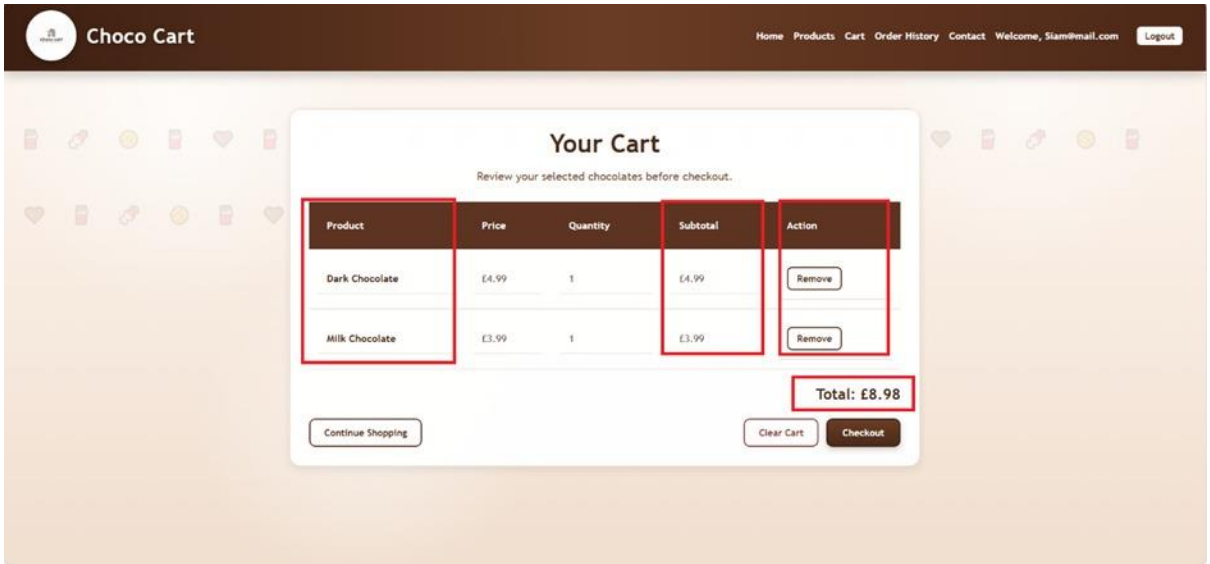



*Fig 5.5.3 – Cart page with items still present after navigating away and back – CartBean session preserved*

## 5.6 UI Components

JSF provides a rich set of server-side UI components rendered as standard HTML. The following JSF components are used across the Choco Cart pages:

| JSF Component | HTML Output | Used On | Purpose |
|---|---|---|---|
| <h:inputText> | <input type="text"> | register.xhtml, login.xhtml, contact.xhtml | Text input for name, email, billing address |
| <h:inputSecret> | <input type="password"> | register.xhtml, login.xhtml | Password fields – value never shown in browser source |
| <h:commandButton> | <input type="submit"> | All form pages | Form submission trigger for login, register, payment, remove, clear cart |
| <h:outputText> |  | All pages | Renders dynamic text – welcome message, error messages, prices, stock levels |
| <h:dataTable> | <table> | cart.xhtml, admin.xhtml | Tabular data display (cart items, admin order list) |

| | | | |
|---|---|---|---|
| <h:panelGroup> | / | cart.xhtml, nav bar | Conditional rendering groups – show/hide based on session state |
| <h:link> | <a> | Navbar on all pages | Client-side navigation between pages via JSF outcome |
| <h:button> | <button> | cart.xhtml | Non-submit navigation button (Continue Shopping) |
| <f:ajax> | JavaScript (auto) | cart.xhtml | Partial page refresh on quantity update, remove, and clear cart |
| <h:messages> | <ul> of messages | cart.xhtml | Renders all FacesMessages in scope (Ajax-updated) |

**register.xhtml – key UI component code example (inputText, inputSecret, commandButton, outputText):**

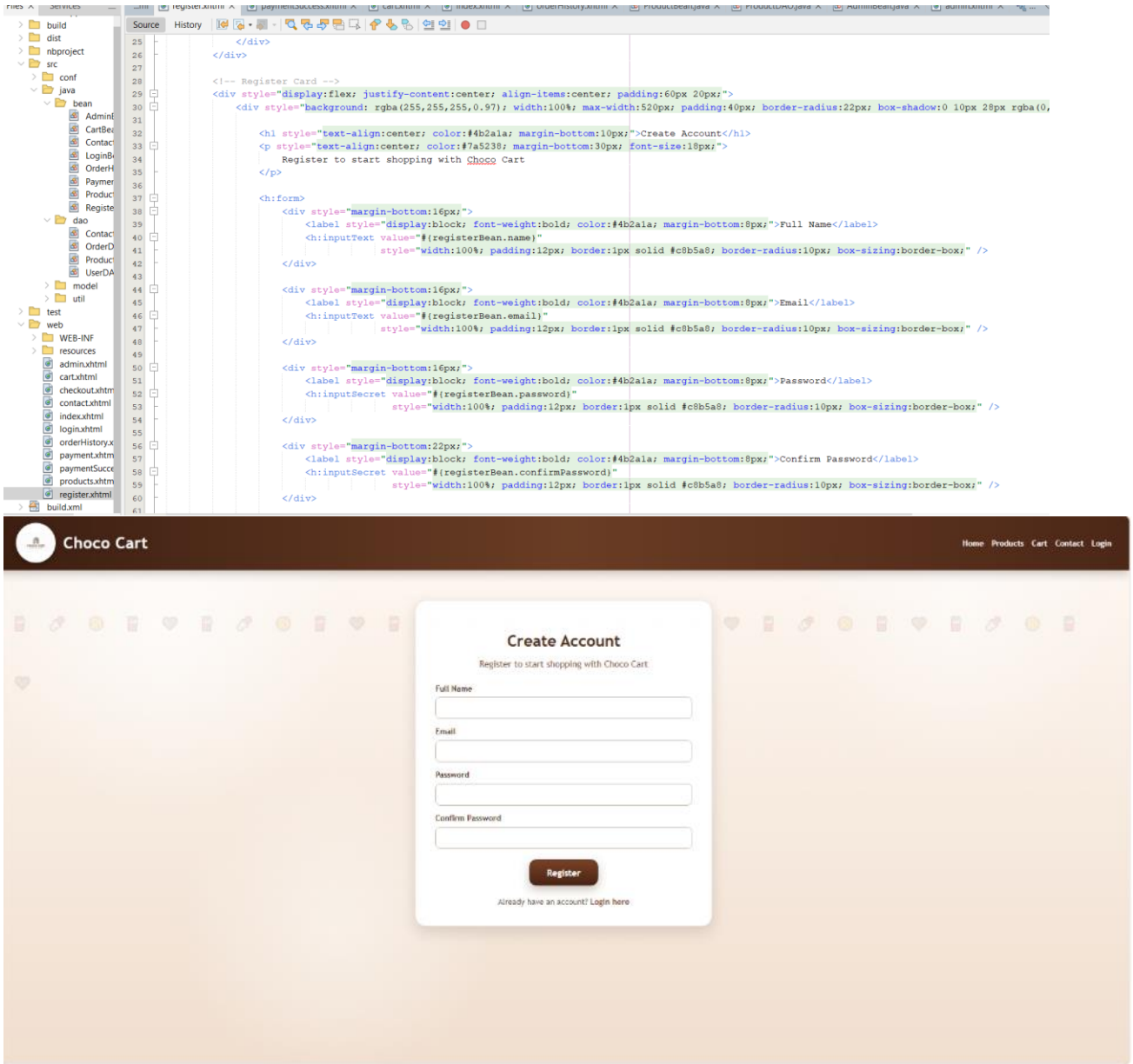



*Fig 5.6.1 – Register page screenshot showing rendered UI (text inputs, password fields, submit button)*

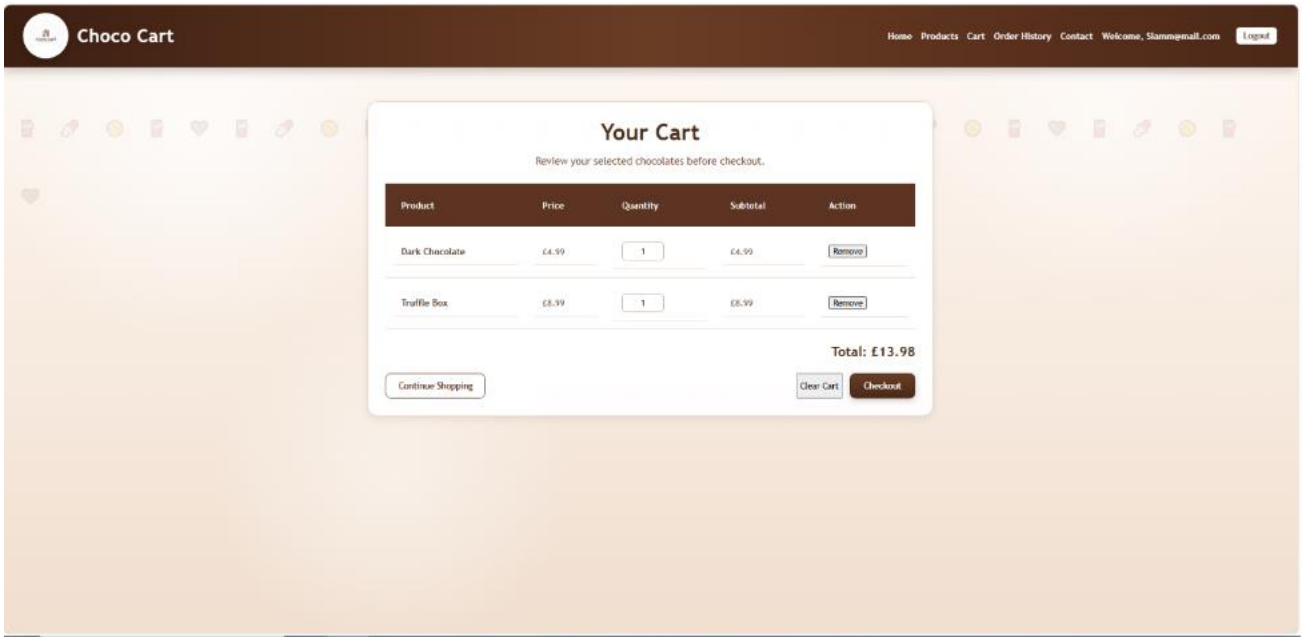



*Fig 5.6.2 – Cart page screenshot showing h:dataTable, h:inputText (quantity), commandButton (Remove), h:panelGroup, and total panel*

## 5.7 Checkout and Payment Flow

The checkout process is protected by a login guard inside PaymentBean.processPayment(). If loginBean.isLoggedIn() is false, the user is redirected to the login page before any payment processing begins.

On successful payment: stock is reduced (UPDATE), a new order is saved (INSERT into ORDERS and ORDER_ITEMS), the cart is cleared (clearCart()), and the user is redirected to the payment success page.

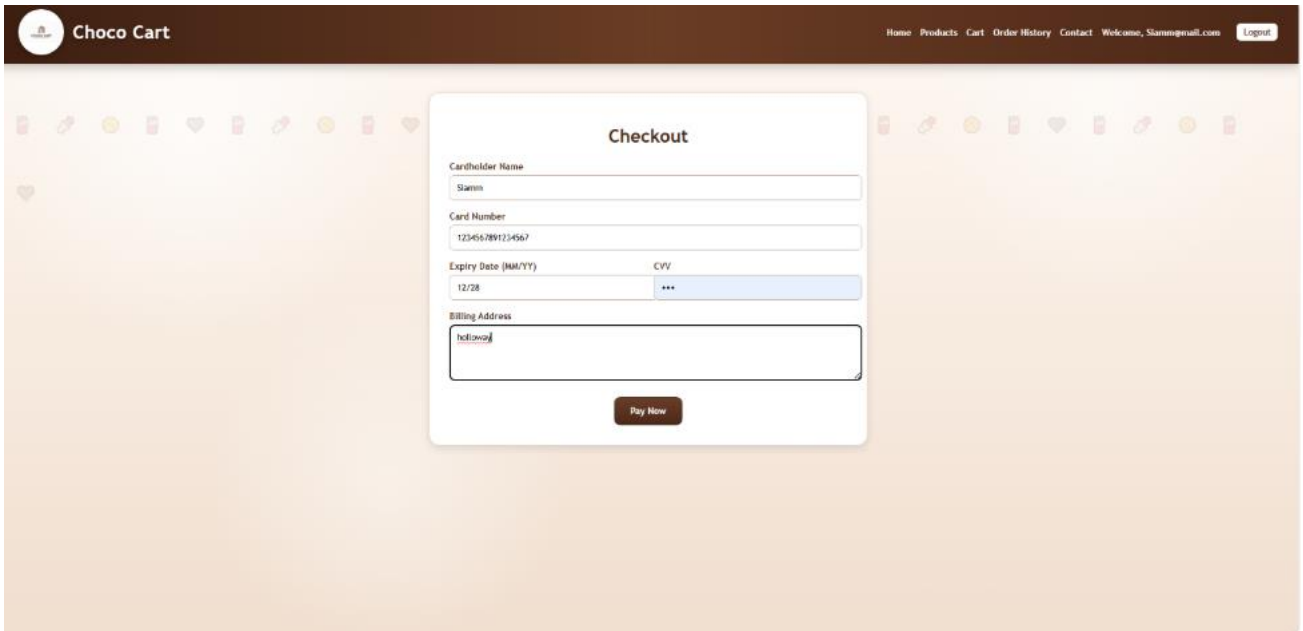



*Fig 5.7.1 – Checkout page with all fields filled (test card: 1234567891234567, CVV: 123, Expiry: 12/28)*

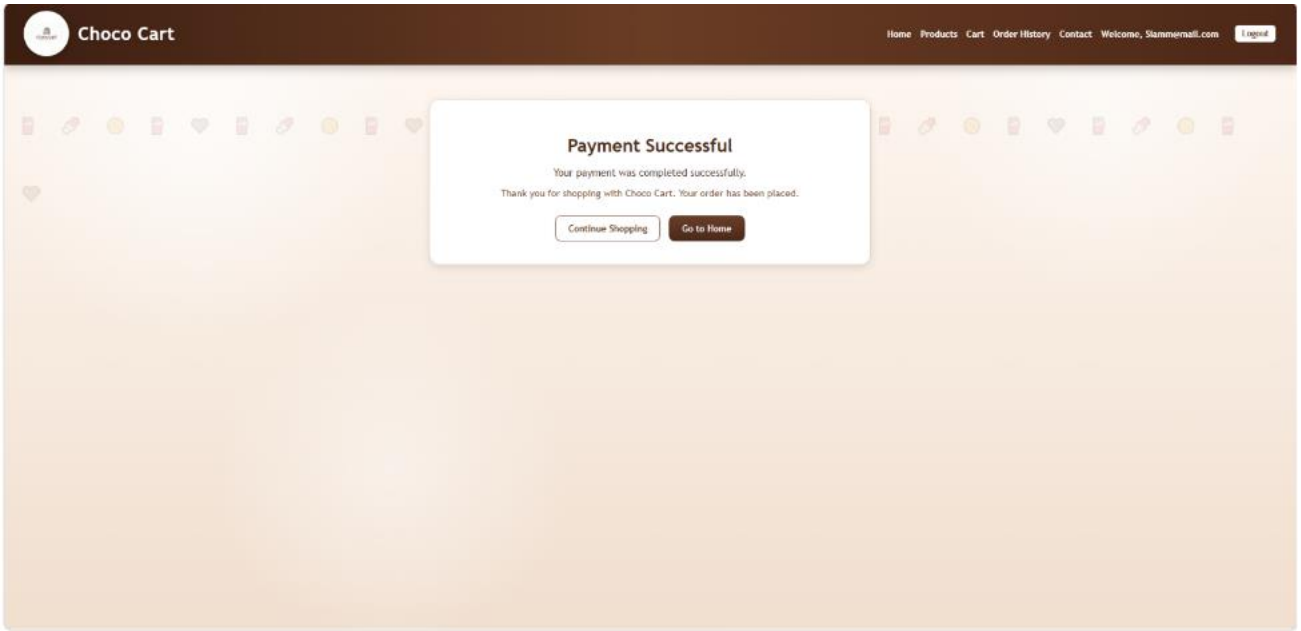



*Fig 5.7.2 – Payment Success page after successful submission*

## 5.8 Order History

The Order History page is only accessible to logged-in users. OrderHistoryBean retrieves all orders where ORDERS.USER_ID = loginBean.userId, displaying order ID, date, and total. This demonstrates a Read with a WHERE clause filtering by session data.

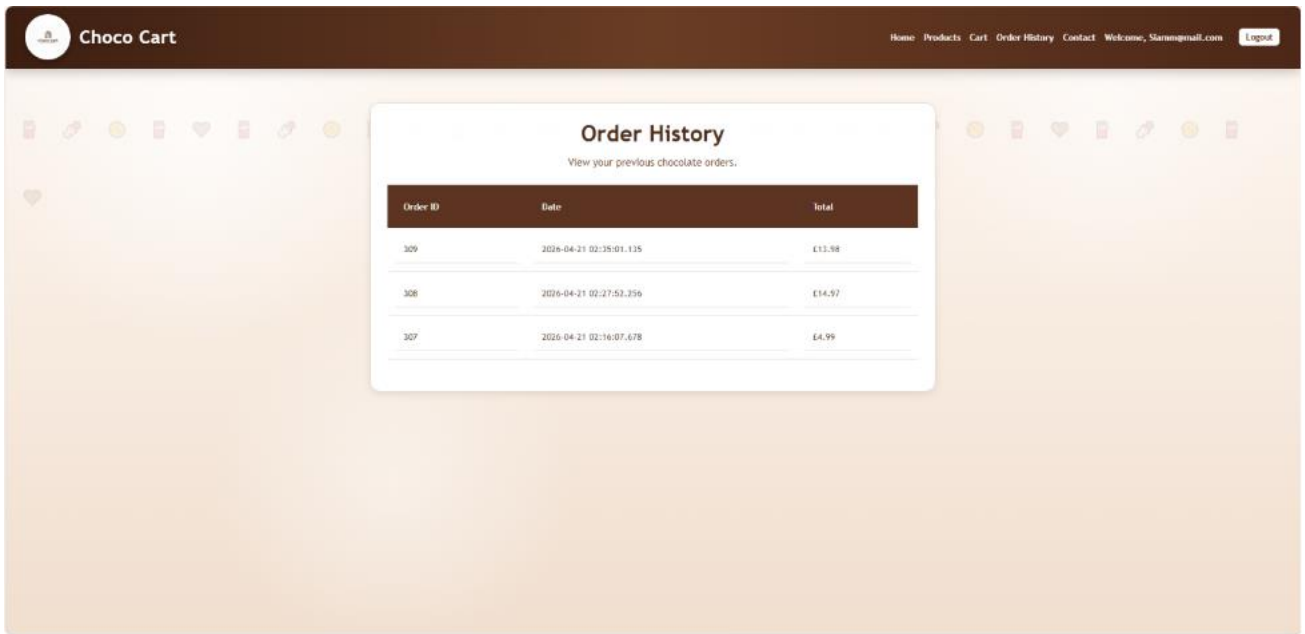
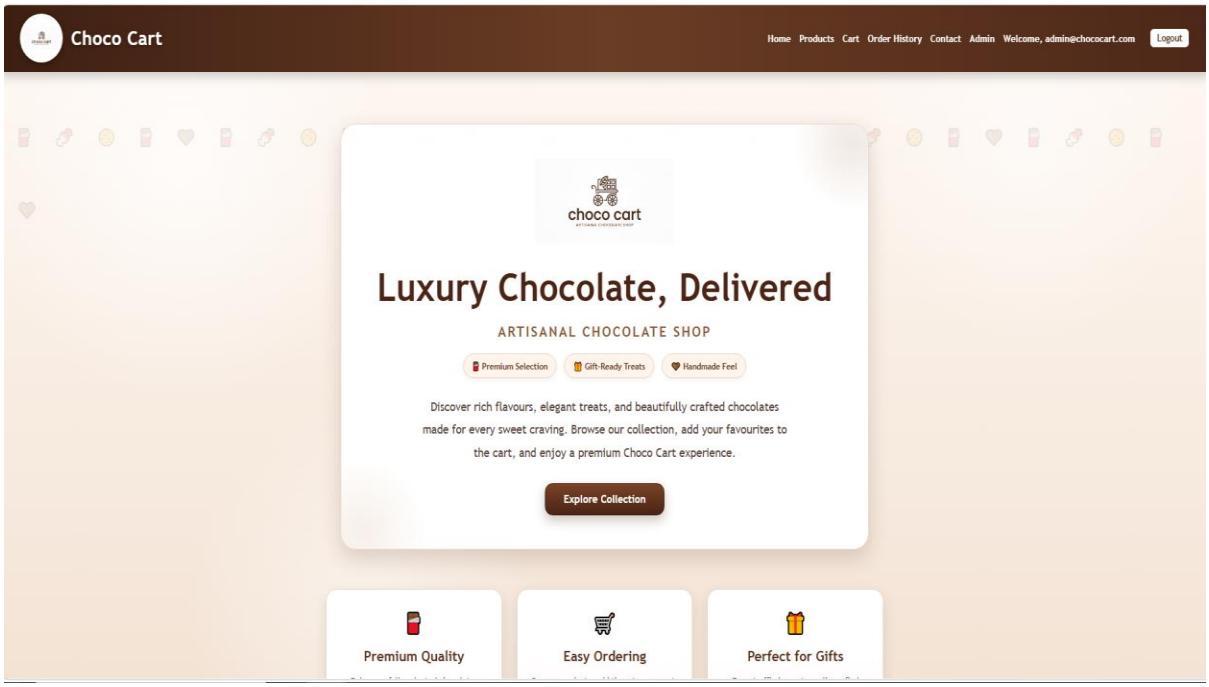
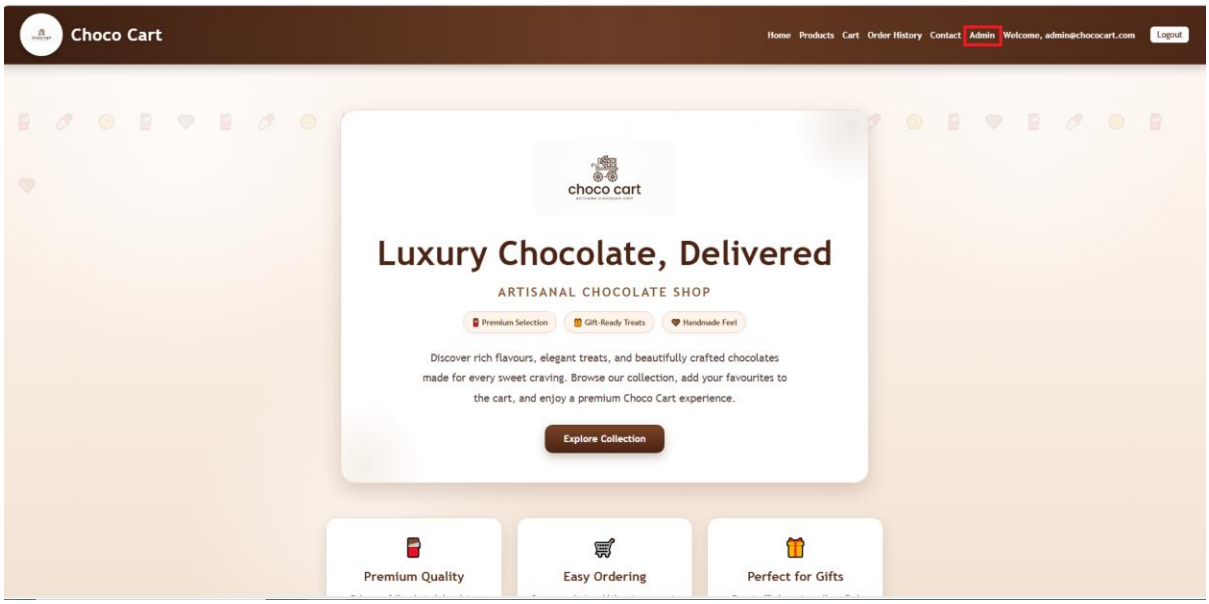



*Fig 5.8.1 – Order History page showing past orders for the logged-in test user*

# 6. ADDITIONAL FEATURES:

## Admin Page:




When a user logs in using the admin email, the system checks the email in `Login Bean` and sets the admin flag to true. This gives the user access to the admin page through the navigation bar.

The admin page allows access to the order of records stored in the database. From there, the admin can view all customer orders, including the order ID, user ID, order date, and total amount. This gives a simple overview of purchase activity and helps the shop monitor customer transactions.

# 7. WORK MANAGEMENT and REFLECTION

## 7.1 Work distribution

This project had a team of three Krish, Aavash, and Sandesh who were assigned different tasks.

| Member | Primary role | Further task |
|---|---|---|
| Krish | Back development/ database/testing | Database design and setup, JDBC connectivity, SQL requirements, and overall testing. |
| Aavash | Frontend/ UI development /testing | JSF page, styling, navigation structure, documentation and testing |
| Sandesh | Testing/ JavaBeans implementation | Testing all the conditions and managed beans for the product. |

All the members contributed to the test throughout the project. This helped to debug the integration issues early. Code comments and the regular updates and handover via the what's app reflect each tram member transparency in the project. The work was distributed fairly and clear responsivities. All in all, the team is satisfied with everyone's effort.

## 7.2. Refection

**- Experience and process.** This project gave us the opportunity to work as a team with hands-on experience. This refined our java web stack understanding like JSF, JavaBeans, JDBC, and session management. We learn how to work in the parallel development of the project with a clear set of tasks and when to interchange the workflow. This project was this team's second project which further helped the process while reflecting the mistake that we made in the last project as a team.

**- Challenges and debugs.** The most major issue that we faced with this project was the session tracking and front-end integration. This task was not a new topic to us but considering the errors while redirecting in JSF, it slowed down our project. These errors were soon resolved by storing basket data in the session before authentication. The team worked together for cross-referencing JSF life cycle documentation. It was pretty helpful in the long run.

-**Lessons learned.**

Define the logic of the session scope and login redirected early. Late changes affect multiple pages and classes.

Share testing is always handy to catch bugs.

Focusing on the simple, clean UI makes sense and makes the workflow faster.

-**CPD contribution.** This project helps everyone with the practical understanding of three-tier architecture, MVC design, and full-stack Java web development. This also helps us to develop habits to work and communicate across roles and finish under deadlines. (Oracle, 2014c: BCS, no date)

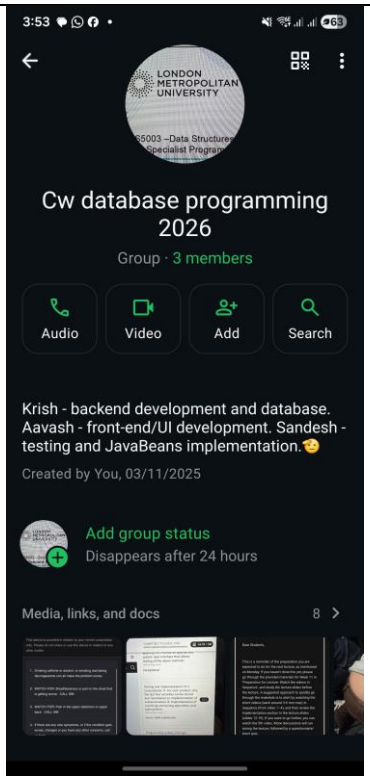



Fig 6.1.0. Screenshot of the group chat on WhatsApp.

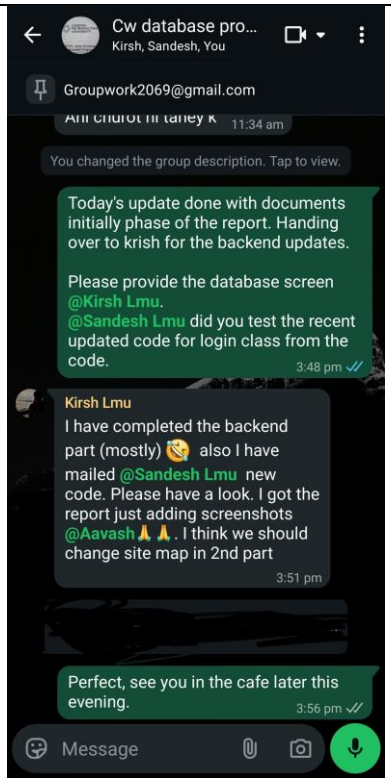



Fig 6.1.1. Screenshot of the group chat handover and updates on WhatsApp.

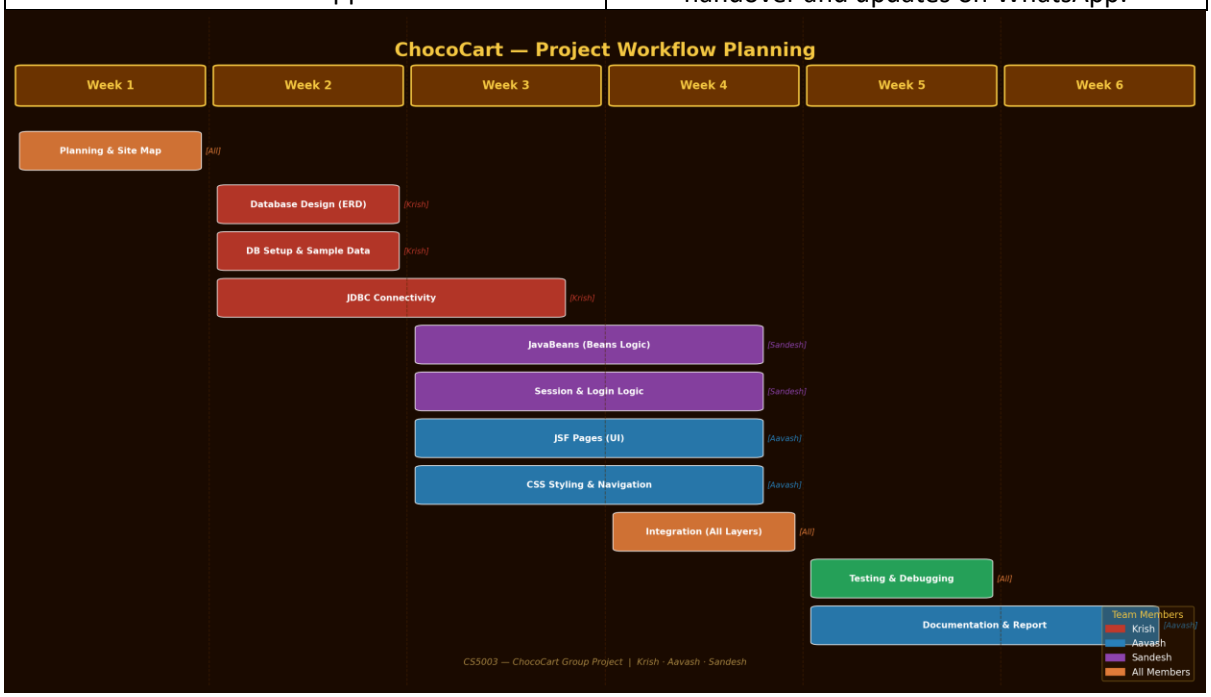



Fig 6.1.2. Choco cart project workflow with the tasks, deadlines, and responsibilities.

# 8. ARTEFACT DESCRIPTION

| File / Group | Technology | Purpose |
|---|---|---|
| index.xhtml | JSF | Main home page of the website. It introduces the shop and provides navigation to the main sections. |
| products.xhtml | JSF | Shows the product catalogue and lets users add chocolates to the cart. |
| cart.xhtml | JSF | Displays items added to the basket, shows the total cost, and allows items to be removed or the cart to be cleared. |
| checkout.xhtml | JSF | Acts as the step between the cart and payment. |
| payment.xhtml | JSF | Collects payment details such as cardholder name, card number, expiry date, CVV, and billing address. |
| paymentSuccess.xhtml | JSF | Confirms that payment has been completed and the order has been placed. |
| login.xhtml | JSF | Lets existing users log in and begin a session. |
| register.xhtml | JSF | Allows new users to create an account with basic validation checks. |
| orderHistory.xhtml | JSF | Shows previous orders for the logged-in user. |
| contact.xhtml | Static page | Provides contact details for the shop and serves as the required static page. |
| style.css | CSS | Controls the visual design of the website, including layout, colours, navigation bar, buttons, and general styling. |
| CartBean.java | Managed Bean | Handles cart actions such as adding products, removing items, clearing the cart, and calculating the total price. |
| LoginBean.java | Managed Bean | Manages user login data, session state, and logout. |
| RegisterBean.java | Managed Bean | Handles account creation and checks for missing fields, password matching, and duplicate email addresses. |
| PaymentBean.java | Managed Bean | Processes payment, updates stock levels, clears the basket, and redirects to the success page. |
| ProductBean.java | Managed Bean | Loads product information from the database for display on the products page. |
| OrderHistoryBean.java | Managed Bean | Retrieves the logged-in user's order history from the database. |
| UserDAO.java | DAO, JDBC | Handles user-related database tasks such as login, registration, and email checking. |
| ProductDAO.java | DAO, JDBC | Retrieves product data and updates stock after purchases. |
| OrderDAO.java | DAO, JDBC | Saves orders, stores order items, and retrieves order history. |

| | | |
|---|---|---|
| DBConnection.java | Utility, JDBC | Provides the database connection used by the DAO classes. |
| User.java, Product.java, CartItem.java, Order.java | Model classes | Represent the main data objects used in the system. |
| web.xml | Jakarta configuration | Sets up the Faces Servlet and define the welcome page. |
| beans.xml | Jakarta configuration | Supports the Jakarta application setup. |
| glassfish-web.xml | GlassFish configuration | Contains deployment settings for running the project on GlassFish. |
| build.xml, project.properties, project.xml | NetBeans / Ant | Support building and running the project on NetBeans. |
| dist/chocoshopp.war | WAR file | Packaged version of the application ready for deployment. |
| resources/images/* | Static resources | Stores the logo and product images used throughout the site. |

| Technology | Purpose |
|---|---|
| **Java** | Used to write the application logic, model classes, and database code. |
| **JSF** | Used to build dynamic pages and interface components. |
| **Managed Beans** | Link the JSF pages to the backend logic and store request or session data. |
| **JDBC** | Connects the application to the database and runs SQL queries. |
| **DAO pattern** | Keeps database code separate from the page and bean logic. (Eclipse Foundation, 2022) |
| **Model classes** | Store data such as users, products, cart items, and orders in object form. |
| **Session tracking** | Keeps the user logged in and preserves cart data while moving between pages. |
| **CSS** | Improves the appearance and consistency of the website. |
| **GlassFish** | Runs the web application on the server side. |
| **NetBeans / Ant** | Used for development, build, and packaging. (Apache, 2024) |

# 9. Reference

Apache Software Foundation (2014) *Apache NetBeans*. Available at: https://netbeans.apache.org/ (Accessed: 18 April 2026).

BCS – The Chartered Institute for IT (no date) *Continuing Professional Development (CPD)*. Available at: https://www.bcs.org/membership-and-registrations/member-benefits/cpd/ (Accessed: 21 April 2026).

Eclipse Foundation (2022) *Eclipse GlassFish 7: Jakarta EE 10 Application Server*. Available at: https://glassfish.org/ (Accessed: 2 April 2026).

Eclipse Foundation (2022) *Jakarta Faces 4.0 Specification*. Available at: https://jakarta.ee/specifications/faces/4.0/ (Accessed: 20 April 2026).

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', in *Proceedings of the ACM CHI'94 Conference on Human Factors in Computing Systems*, Boston, MA, 24–28 April, pp. 152–158. Available at: https://doi.org/10.1145/191666.191729 (Accessed: 18 April 2026).

Oracle (2014a) *Using Ajax with JavaServer Faces Technology*. In: *The Java EE 7 Tutorial, Release 7*. Oracle Corporation. Available at: https://docs.oracle.com/javaee/7/tutorial/jsf-ajax.htm (Accessed: 20 April 2026).

Oracle (2014b) *Maintaining Client State: Session Tracking*. In: *The Java EE 7 Tutorial, Release 7*. Oracle Corporation. Available at: https://docs.oracle.com/javaee/7/tutorial/servlets009.htm (Accessed: 2 April 2026).

Oracle (2014c) *Overview: Developing with Java EE — The MVC Design Pattern*. In: *The Java EE 7 Tutorial, Release 7*. Oracle Corporation. Available at: https://docs.oracle.com/javaee/7/tutorial/overview004.htm (Accessed: 2 April 2026).

Oracle (2024a) *Trail: JDBC Database Access*. In: *The Java™ Tutorials*. Oracle Corporation. Available at: https://docs.oracle.com/javase/tutorial/jdbc/ (Accessed: 1 April 2026).

Oracle (2024b) *JavaBeans*. In: *The Java™ Tutorials*. Oracle Corporation. Available at: https://docs.oracle.com/javase/tutorial/javabeans/ (Accessed: 1 April 2026).